UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA — UTEC

Proyecto 1: Tiendas en Línea Falabella Perú



Base de Datos I

Profesor: **Wilder Nina**

Integrantes:

Nombre y Apellidos	**Código**
Adrian Rojas Tejada	202510038
Gabriel Saavedra Peralta	202510192
Sebastian Perez Carrasco	202510248

Ciclo 2026-1

Índice

1. Requisitos **3**
- 1.1. Introducción . . . 3
- 1.2. Descripción general del problema/organización/empresa . . . 3
- 1.3. Necesidad/usos de la base de datos . . . 3
- 1.4. ¿Cómo resuelve el problema hoy? . . . 4
  - 1.4.1. ¿Cómo se almacenan/procesa los datos hoy? . . . 4
  - 1.4.2. Flujo de datos . . . 5
- 1.5. Descripción detallada del sistema . . . 5
  - 1.5.1. Objetos de información actuales . . . 5
  - 1.5.2. Características y funcionalidades esperadas . . . 6
  - 1.5.3. Tipos de usuarios existentes/necesarios . . . 7
  - 1.5.4. Tipos de consulta, actualizaciones . . . 7
  - 1.5.5. Tamaño estimado de la base de datos . . . 8
- 1.6. Objetivos del proyecto . . . 9
  - 1.6.1. Objetivo Principal . . . 9
  - 1.6.2. Objetivos Secundarios . . . 9
- 1.7. Referencias del proyecto . . . 9
- 1.8. Eventualidades . . . 9
  - 1.8.1. Problemas que pudieran encontrarse en el proyecto . . . 9
  - 1.8.2. Límites y alcance del proyecto . . . 9

2. Modelo Entidad-Relación **11**
- 2.1. Reglas semánticas . . . 11
- 2.2. Modelo Entidad-Relación . . . 11
- 2.3. Especificaciones y consideraciones sobre el modelo . . . 12
  - 2.3.1. Entidad Clientes . . . 12
  - 2.3.2. Entidad Categorias . . . 13
  - 2.3.3. Entidad Vendedores . . . 13
  - 2.3.4. Entidad Productos . . . 13
  - 2.3.5. Entidad Ordenes . . . 14
  - 2.3.6. Entidad DetalleOrden . . . 14
  - 2.3.7. Entidad Pagos . . . 14
  - 2.3.8. Entidad Envios . . . 14
  - 2.3.9. Entidad Resenas . . . 15
  - 2.3.10. Entidad Cupones . . . 15

3. Modelo Relacional **15**
- 3.1. Modelo Relacional . . . 15
- 3.2. Especificaciones de transformación . . . 16
  - 3.2.1. Entidades regulares . . . 16
  - 3.2.2. Entidades débiles . . . 17
  - 3.2.3. Entidades superclase/subclase . . . 17
  - 3.2.4. Relaciones binarias . . . 18

3.2.5. Relaciones ternarias . . . 19
3.3. Diccionario de datos . . . 19

4. Implementación de la base de datos **22**
4.1. Creación de Tablas en PostgreSQL . . . 22
4.2. Carga de datos . . . 27
4.3. Simulación de Datos Faltantes . . . 28

5. Optimización y Experimentación **29**
5.1. Consultas SQL para el experimento . . . 29
5.1.1. Descripción del tipo de consultas seleccionadas . . . 29
5.1.2. Implementación de consultas en SQL . . . 30
5.2. Metodología del experimento . . . 32
5.3. Optimización de consultas . . . 33
5.3.1. Planes de índices para Consulta 1 . . . 33
5.3.2. Planes de índices para Consulta 2 . . . 34
5.3.3. Planes de índices para Consulta 3 . . . 36
5.3.4. Planes de índices para Consulta 4 . . . 38
5.4. Plataforma de Pruebas . . . 39
5.5. Medición de tiempos . . . 40
5.5.1. Sin índices . . . 40
5.5.2. Con índices . . . 41
5.6. Resultados . . . 42
5.6.1. Consulta 1 . . . 42
5.6.2. Consulta 2 . . . 43
5.6.3. Consulta 3 . . . 44
5.6.4. Consulta 4 . . . 45
5.7. Análisis y Discusión . . . 45
5.7.1. Consulta 1: Análisis Técnico de la Anomalía Semántica e Impacto de *Fan-out* . . . 46
5.7.2. Consulta 2: Comportamiento Asintótico y Degradación por Volumen de Filas . . . 46
5.7.3. Consulta 3: Inocuidad de Índices en Consultas Complejas Acotadas por CPU . . . 47
5.7.4. Consulta 4: Éxito Rotundo del Filtrado Indexado y Alta Selectividad 47

6. Conclusiones **47**

7. Anexos **48**
7.1. Normalización de la base de datos . . . 49
7.2. Dump Data y scripts . . . 50

1. Requisitos

1.1. Introducción

La empresa analizada en este proyecto es Falabella Perú, una plataforma integral de comercio electrónico y soluciones digitales que opera a nivel nacional. Se define como un ecosistema tecnológico masivo diseñado para unificar los canales de venta físicos y virtuales, conectando de forma síncrona a compradores, tiendas asociadas y operadores logísticos.

Esta organización requiere de manera crítica una base de datos relacional para centralizar, proteger y dar coherencia a toda su información comercial de gran volumen. Sin una infraestructura de datos robusta, la empresa no podría mantener la consistencia en el control de sus inventarios dinámicos, procesar miles de transacciones financieras concurrentes por segundo, asegurar la persistencia de datos confidenciales o coordinar las complejas operaciones de distribución, poniendo en riesgo la continuidad del negocio y la confianza del consumidor.

El sustento teórico de este diseño se basa en el Modelo Relacional y la teoría de normalización de bases de datos. Ante la naturaleza transaccional de Falabella Perú, se requiere un motor relacional como PostgreSQL que garantice estrictamente las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad). Esto asegura que operaciones complejas, como el descuento de inventario simultáneo al pago, se ejecuten de forma segura sin corromper la integridad de los datos, superando las limitaciones operativas de los sistemas basados en archivos aislados o enfoques no relacionales.

1.2. Descripción general del problema/organización/empresa

Falabella Perú opera como una gran tienda por departamentos digital bajo un modelo de negocio híbrido y omnicanal. Este modelo combina la venta minorista directa de stock propio (es decir, productos adquiridos, almacenados y distribuidos directamente por la empresa) con un modelo de Marketplace, el cual permite a miles de proveedores y emprendedores externos utilizar la infraestructura de la plataforma para ofrecer sus propios catálogos a cambio de una comisión por venta.

En el contexto del mercado peruano, el sector del e-commerce es sumamente competitivo y centralizado, experimentando picos masivos de demanda durante eventos estacionales como los CyberWow, el Black Friday o campañas navideñas. En este entorno, el problema principal de la organización radica en la dificultad de unificar la gestión logística de distribución a nivel nacional y en mitigar la asimetría de información entre el stock real de los vendedores externos y lo que se muestra en la web, lo cual suele generar quiebres de inventario, retrasos en las entregas y problemas de facturación cruzada.

1.3. Necesidad/usos de la base de datos

Al ser un modelo de negocio híbrido digital, Falabella Perú depende enteramente de la data para operar. En el competitivo mercado del e-commerce nacional, se requiere una infraestructura relacional robusta en PostgreSQL para centralizar la información y garantizar una

experiencia fluida. La base de datos es indispensable para articular de forma segura las siguientes necesidades funcionales del sistema:

Gestión de clientes: Almacena perfiles de usuarios, credenciales cifradas, historiales y múltiples direcciones de entrega para agilizar la compra y personalizar la experiencia.

Catálogo de productos: Organiza categorías, descripciones y el stock físico en tiempo real. Diferencia con precisión el inventario propio del de terceros (Marketplace) para evitar quiebres de stock.

Órdenes de compra: Registra cada pedido vinculando múltiples productos en un carrito, congela los precios históricos de venta para auditorías y realiza la trazabilidad del estado de la orden.

Procesamiento de pagos: Garantiza transacciones seguras almacenando códigos de autorización bancaria y métodos de pago (tarjetas, Yape, Plin) para soportar de manera consistente miles de compras concurrentes.

Control de envíos y logística: Monitorea el flujo de distribución nacional vinculando las órdenes con empresas de transporte (couriers), rutas y números de seguimiento en tiempo real.

Motor de cupones y descuentos: Gestiona las reglas semánticas, vigencia y restricciones de los códigos promocionales para aplicar las rebajas exactas durante el proceso de pago.

Con esta arquitectura, Falabella Perú soluciona los problemas de inconsistencia y quiebres de inventario actuales, transformando el registro transaccional en una ventaja competitiva para la toma de decisiones estratégicas.

1.4. ¿Cómo resuelve el problema hoy?

1.4.1. ¿Cómo se almacenan/procesa los datos hoy?

Actualmente, la empresa opera de forma fragmentada mediante sistemas heredados (legacy) y procesos manuales aislados, lo que impide la sincronización en tiempo real:

Inventario y Marketplace: El stock propio reside en un sistema antiguo rígido, mientras que los vendedores externos envían sus actualizaciones en hojas de Excel por correo. Un equipo las consolida manualmente una vez al día, causando desfases y quiebres de stock en la web.

Clientes y Órdenes: Los pedidos se guardan temporalmente en un servidor web básico y se migran en lotes (batches) nocturnos hacia un archivo plano, perdiendo la visibilidad del estado de las compras durante el día.

Pagos y Envíos: La pasarela externa exporta reportes en CSV cada tres horas. Logística descarga estos archivos, cruza los datos manualmente con las órdenes pendientes y genera a mano las etiquetas para los couriers.

Este entorno descentralizado genera alta redundancia de datos, lentitud operativa y errores manuales de digitación.

1.4.2. Flujo de datos

Para visualizar de manera clara el trayecto actual de la información, se ha elaborado un diagrama de flujo de datos estructurado en la plataforma Lucidchart (herramienta provista por la universidad). Este modelo describe la secuencia lineal e interactiva del proceso transaccional estándar del e-commerce:

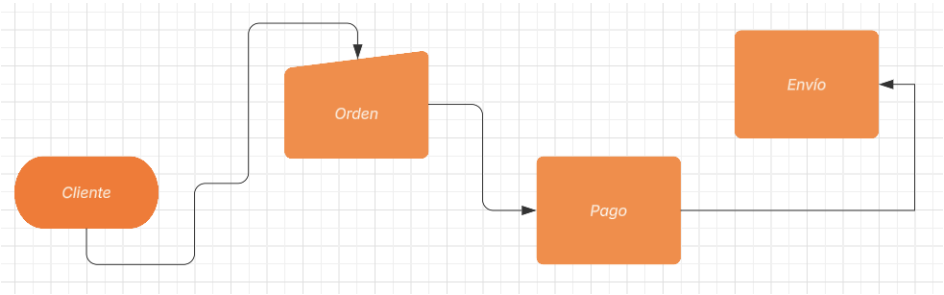


Figura 1: Flujo de Datos

Cliente → Registro de Orden: El proceso inicia cuando el Cliente interactúa con la interfaz web y añade productos al carrito, enviando una petición de compra que genera una Orden con estado Pendiente en el servidor.

Orden → Procesamiento de Pago: Los datos de la orden y el monto total son redirigidos de manera síncrona hacia la entidad de Pago, la cual valida las credenciales bancarias del usuario a través de la pasarela de pagos externa.

Pago → Verificación y Confirmación: Una vez que la transacción financiera es procesada por la pasarela externa, esta no se refleja inmediatamente. El sistema almacena la orden como pendiente hasta recibir el reporte consolidado en CSV (cada 3 horas), lo que genera un desfase crítico donde el stock no se descuenta al momento, propiciando la sobreventa de productos.

Orden → Gestión de Envío: Finalmente, los datos de los pedidos pagados son extraídos manualmente al final del día por el equipo de logística. Estos se cruzan de forma manual con las direcciones de los compradores para generar las etiquetas de despacho para los couriers, perdiéndose la capacidad de ofrecer un número de seguimiento en tiempo real al cliente.

1.5. Descripción detallada del sistema

1.5.1. Objetos de información actuales

Para estructurar la base de datos, se han definido los siguientes 10 objetos de información o entidades del sistema:

1. Clientes: Almacena los datos personales (DNI/CE, nombres, apellidos), información de contacto (teléfono, correo electrónico) y credenciales de acceso de los usuarios compradores.

2. Productos: Registra el catálogo de artículos disponibles en la plataforma con su código único (SKU), nombre, descripción técnica, marca y precio base.

3. Órdenes: Modela el encabezado de cada pedido realizado por un cliente, guardando el identificador único, la fecha de la compra y el estado global de la solicitud.

4. DetalleOrden: Relación específica de los artículos incluidos en una orden, registrando la cantidad solicitada de cada producto y congelando su precio unitario en el instante exacto de la venta.

5. Pagos: Contiene la información financiera de la transacción, incluyendo el método de pago utilizado (tarjeta, Yape, Plin), la fecha de procesamiento y el código de autorización de la pasarela.

6. Envíos: Registra los datos de distribución logística del pedido, tales como la dirección física de destino, la empresa de transporte (courier) asignada y la fecha estimada de entrega.

7. Categorías: Clasifica jerárquicamente el catálogo de productos (por ejemplo: Tecnología, Moda, Hogar) para estructurar el menú de navegación del sitio web.

8. Vendedores: Almacena los datos de los comercios y proveedores externos asociados al modelo de Marketplace (RUC, razón social y calificación).

9. Reseñas: Guarda las calificaciones cuantitativas (estrellas de 1 a 5) y los comentarios escritos que los clientes dejan sobre los productos que han adquirido.

10. Cupones: Registra los códigos alfanuméricos promocionales de descuento, especificando su fecha de vigencia, porcentaje o monto de rebaja, y límites de uso.

1.5.2. Características y funcionalidades esperadas

El sistema propuesto en PostgreSQL debe ser altamente eficiente, seguro y escalable, garantizando un comportamiento óptimo a medida que el volumen de datos escale desde miles hasta un millón de registros según los objetivos de la experimentación del proyecto. A continuación, se detallan las características y funcionalidades principales integrando los tiempos de respuesta y niveles de servicio esperados:

- **Búsqueda y filtrado por categoría:** Permitir a los clientes indexar, filtrar y explorar el catálogo multicategoría de forma dinámica. Debido a que es la operación de consulta más frecuente por los usuarios, se espera que el sistema resuelva las búsquedas mediante índices avanzados en un tiempo de respuesta menor a **100 ms**, incluso bajo escenarios de alta concurrencia masiva con un volumen de hasta 1,000,000 de productos.
- **Gestión del carrito de compras:** Capacidad de registrar, actualizar y mantener de forma consistente los productos seleccionados por el cliente antes de proceder a la compra. El sistema debe asegurar que añadir o remover elementos sea una modificación ágil que no bloquee las tablas principales, manteniendo un tiempo de procesamiento inmediato.
- **Procesamiento de Checkout (Orden y Pago):** Es la funcionalidad transaccional más crítica del e-commerce. Debe ejecutar de forma atómica (propiedades ACID) la creación del registro en la tabla *Órdenes*, la inserción de múltiples tuplas en *DetalleOrden*, la validación del cobro en *Pagos* y la deducción automática del stock físico en

Productos. Para evitar problemas de sobreventa en eventos como el *CyberWow*, esta operación compuesta debe completarse de forma segura en un lapso menor a **500 ms**.

- **Consulta de historial de órdenes:** Permitir a los usuarios y al personal de atención al cliente acceder al registro histórico de todas las compras asociadas a un identificador. Esta operación involucra reuniones (*JOINs*) complejas entre las tablas *Clientes*, *Órdenes* y *DetalleOrden*, por lo que el diseño de llaves primarias y foráneas debe garantizar la extracción del reporte en menos de **200 ms**.
- **Motor de aplicación de cupones:** Validación en tiempo real de las reglas semánticas de los códigos promocionales introducidos en el checkout. La base de datos debe verificar instantáneamente las restricciones de vigencia, tipo de descuento y que la cantidad en *usos_actuales* sea estrictamente menor a *usos_maximos*, aplicando el descuento de manera exacta antes de consolidar el pago.
- **Seguimiento de envíos (Tracking):** Proveer actualizaciones constantes sobre el estado logístico del pedido a través de la entidad *Envíos*. Esta consulta es realizada de forma interactiva tanto por el cliente final para visualizar su número de seguimiento como por las empresas de transporte (*couriers*) para registrar las confirmaciones de entrega en la base de datos.

1.5.3. Tipos de usuarios existentes/necesarios

- **Cliente:** Usuario final que navega por el catálogo, gestiona su carrito, aplica cupones y realiza compras.
- **Vendedor Externo:** Proveedor externo que gestiona su propio inventario, publica productos y revisa sus ventas.
- **Administrador:** Encargado del mantenimiento global, supervisión de usuarios, moderación de reseñas y gestión de categorías.
- **Sistema de pagos (Pasarela):** Usuario a nivel de sistema que confirma o rechaza las transacciones económicas de las órdenes.

1.5.4. Tipos de consulta, actualizaciones

Consultas que podrían hacer trabajadores y usuarios:

- ¿Qué productos tienen más devoluciones o cancelaciones?
- ¿Cuál es el vendedor externo con más ventas en el mes?
- ¿Qué cupones de descuento están por vencer en los próximos días?

Actualizaciones que tendrían los usuarios:

- Alta de productos de manera diaria al catálogo por parte de los vendedores externos.
- Cambio de precios y actualizaciones críticas de stock en tiempo real tras concretarse una transacción.

- Cancelación de órdenes y modificación manual del estado de los envíos por cuestiones logísticas o reclamos.

1.5.5. Tamaño estimado de la base de datos

Para estimar el tamaño de la base de datos nos guiaremos de las tablas del modelo relacional donde se han encontrado entidades que a lo largo del tiempo serán fijas y otras entidades que cambiarán constantemente. Primero, calcularemos la longitud de todas las entidades en bytes y luego obtendremos el tamaño esperado total de las entidades fijas y cambiantes.

Nombre de tabla	**Longitud de atributos (bytes)**	**Longitud total (bytes)**
Clientes	$4 + 100 + 100 + 150 + 20 + 4 + 8$	386
Productos	$4 + 200 + 500 + 8 + 4 + 100 + 30 + 4 + 4$	854
Órdenes	$4 + 8 + 30 + 8 + 4 + 4$	58
DetalleOrden	$4 + 4 + 4 + 8 + 8$	28
Pagos	$4 + 50 + 30 + 8 + 8 + 50 + 4$	154
Envíos	$4 + 300 + 30 + 4 + 4 + 100 + 50 + 4$	496
Categorías	$4 + 80 + 200$	284
Vendedores	$4 + 150 + 150 + 11 + 4$	319
Reseñas	$4 + 4 + 1000 + 4 + 4 + 4$	1020
Cupones	$4 + 30 + 8 + 20 + 4 + 4 + 4$	74

Tabla 1: Longitud calculada de las tablas de E-commerce

Nombre de tabla	**Tamaño en bytes**	**Datos estimados**	**Tamaño total bytes**
Productos	854	80,000	68,320,000
Categorías	284	50	14,200
Vendedores	319	5,000	1,595,000
Cupones	74	500	37,000

Tabla 2: Tamaños de las tablas fijas

TOTAL (Tablas fijas) = 69,966,200 bytes

Nombre de tabla	**Tamaño en bytes**	**Crecimiento por día**	**Crecimiento anual**	**Total por año bytes**
Clientes	386	1,370	500,000	193,000,000
Órdenes	58	5,479	2,000,000	116,000,000
DetalleOrden	28	16,438	6,000,000	168,000,000
Pagos	154	5,479	2,000,000	308,000,000
Envíos	496	4,931	1,800,000	892,800,000
Reseñas	1020	2,739	1,000,000	1,020,000,000

Tabla 3: Tamaños de las tablas cambiantes

TOTAL (Tablas cambiantes) = 2,697,800,000 bytes

1.6. Objetivos del proyecto

1.6.1. Objetivo Principal

Diseñar e implementar un modelo de base de datos relacional robusto y eficiente para una plataforma de e-commerce multicategoría que optimice el flujo transaccional de compras y garantice el correcto procesamiento de la información.

1.6.2. Objetivos Secundarios

- Aplicar lo aprendido en el curso frente a un contexto realista.
- Implementar restricciones de integridad y de dominio (*Primary Keys, Foreign Keys, Check Constraints*) para garantizar la consistencia y validez de la información registrada en el sistema.
- Elaborar consultas SQL eficientes que involucren múltiples tablas y funciones de agregación, permitiendo simular la extracción de reportes y métricas relevantes para el negocio.

1.7. Referencias del proyecto

- **Falabella.com:** Análisis del modelo de negocio, estructura de catálogo multi-vendedor (Marketplace) y flujos logísticos de cara al cliente final.
- **Documentación oficial de PostgreSQL:** Referencia técnica fundamental para la selección óptima de tipos de datos, uso de restricciones transaccionales ACID e índices de rendimiento.
- **Datasets de e-commerce peruano:** Datos e históricos estadísticos extraídos del comercio electrónico local para aproximar de manera realista el volumen de tuplas, crecimiento anual y comportamiento de operaciones de pago.

1.8. Eventualidades

1.8.1. Problemas que pudieran encontrarse en el proyecto

Uno de los problemas sería la sobreventa de inventario durante campañas masivas si los bloqueos de transacciones no se manejan correctamente, causando que la cantidad vendida no concuerde con el stock real. Además, podrían existir inconsistencias por pagos fallidos en la comunicación con la pasarela.

1.8.2. Límites y alcance del proyecto

Límites: Uno de los problemas sería que los vendedores externos registren mal su stock físico o que los productos se pierdan o dañen durante el envío. De esta manera, no concordaría la cantidad vendida en la plataforma web con la disponible realmente en los almacenes de los proveedores.

Alcance: El alcance sería organizar de manera óptima la base de datos para el funcionamiento de una tienda virtual multicategoría, permitiendo una mejor organización de las órdenes de compra, el control de las ventas de los vendedores y el procesamiento de las reseñas de los clientes.

2. Modelo Entidad-Relación

2.1. Reglas semánticas

1. Un cliente se identifica por un `id_cliente` único y posee nombre, apellido, correo electrónico, teléfono y fecha de registro. Un cliente puede realizar cero o más órdenes.
2. Cada orden pertenece a exactamente un cliente y contiene uno o más detalles de orden. Una orden tiene fecha, estado y un monto total calculado.
3. `DetalleOrden` es una entidad débil de `Ordenes`. Su llave primaria es la combinación del identificador de la orden y el identificador del producto. Almacena la cantidad solicitada, el precio unitario en el momento de la compra y el subtotal.
4. Cada orden genera exactamente un pago y cero o un envío. Un pago y un envío pertenecen a exactamente una orden.
5. Un producto pertenece a exactamente una categoría. Una categoría puede agrupar cero o más productos.
6. Cada producto es ofrecido por exactamente un vendedor. Un vendedor puede ofrecer uno o más productos.
7. Un cliente puede escribir cero o más reseñas. Cada reseña está vinculada a una línea de compra específica (`DetalleOrden`), calificando al producto comprado dentro de una orden determinada.
8. Una orden puede tener aplicado cero o un cupón. Un cupón puede aplicarse a cero o más órdenes. Un cupón no puede utilizarse si ha superado su límite de usos o si su fecha de vencimiento ha pasado.
9. El stock de un producto se descuenta únicamente al confirmarse el pago de la orden, no al agregar el producto al carrito.
10. El precio unitario almacenado en `DetalleOrden` refleja el precio del producto en el momento de la compra y es independiente de cambios posteriores en el precio del producto.
11. Un vendedor se identifica por su RUC, el cual es único en el sistema. Un vendedor puede ser tanto una empresa como una persona natural.
12. La puntuación de una reseña es un entero entre 1 y 5 inclusive. Un cliente solo puede escribir una reseña por producto por orden completada.

2.2. Modelo Entidad-Relación

El diagrama del modelo Entidad-Relación se presenta en la Figura 2.

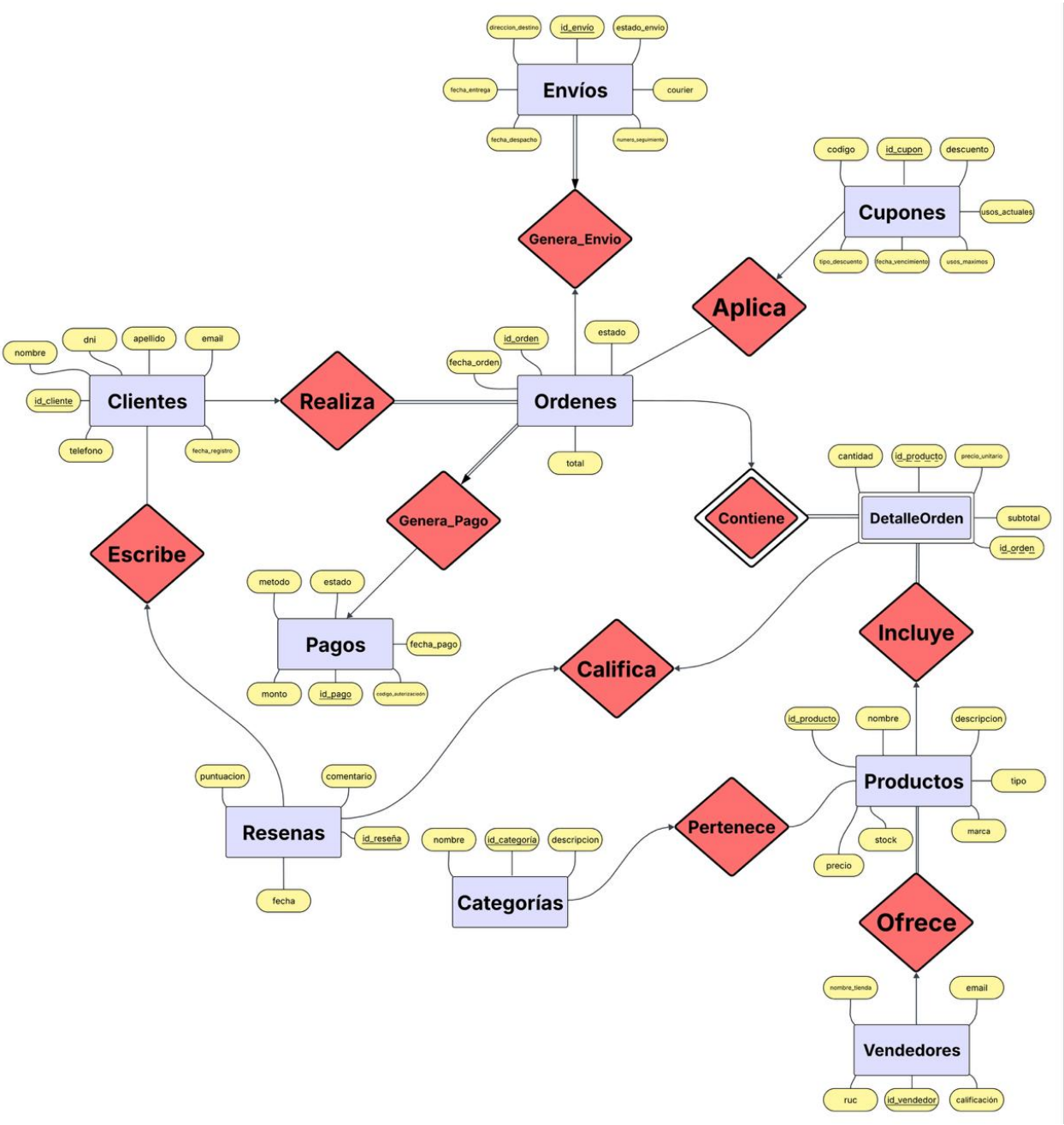


Figura 2: Diagrama del modelo Entidad-Relación

2.3. Especificaciones y consideraciones sobre el modelo

2.3.1. Entidad Clientes

Especificaciones

Almacena la información de cada comprador registrado en la plataforma. La llave primaria es `id_cliente` (INTEGER), un identificador numérico autoincremental.

Consideraciones

Se usa un número como identificador principal en vez del correo porque el correo puede cambiar y causar problemas en otras tablas. La contraseña no se guarda directamente por seguridad. La dirección de envío se guarda en *Envíos* y no en *Clientes*, ya que cada pedido puede enviarse a una dirección distinta. Se incluye `dni` (CHAR(8)) como documento de identidad obligatorio en el contexto peruano, llave candidata única por restricción legal.

2.3.2. Entidad Categorias

Especificaciones

Almacena las categorías del catálogo de productos, como *Electrónico*, *Ropa* o *Hogar*. La llave primaria es `id_categoria` (INTEGER).

Consideraciones

Se usa un identificador numérico en lugar del nombre porque los nombres de las categorías pueden cambiar. La descripción es opcional, ya que algunas categorías no necesitan información adicional. Esto ayuda a mantener la base de datos organizada.

2.3.3. Entidad Vendedores

Especificaciones

Almacena la información de los vendedores que publican productos en la plataforma, tanto la tienda propia de Falabella como vendedores externos del marketplace. La llave primaria es `id_vendedor` (INTEGER).

Consideraciones

El atributo `ruc` es único y no puede ser nulo, pero se usa `id_vendedor` como llave primaria para simplificar las referencias en `Productos`. El atributo `calificacion` es un promedio de las reseñas de los productos del vendedor; se guarda directamente para evitar calcular el promedio en cada consulta.

2.3.4. Entidad Productos

Especificaciones

Almacena el catálogo completo de artículos disponibles para la venta. La llave primaria es `id_producto` (INTEGER). Contiene llaves foráneas a `Categorias` y `Vendedores`.

Consideraciones

El atributo `precio` representa el precio de venta actual. El precio al momento de la compra se guarda en el atributo `precio_unitario` de `DetalleOrden`, evitando que cambios futuros modifiquen el registro de órdenes pasadas. El atributo `stock` no puede ser negativo; su actualización se controla al confirmar el pago para evitar errores. Se añaden `marca` para facilitar búsquedas por fabricante y `tipo` para distinguir productos físicos de digitales, implementando la decisión de diseño descrita en la sección de superclase/subclase.

2.3.5. Entidad Ordenes

Especificaciones

Almacena cada transacción de compra iniciada por un cliente. La llave primaria es `id_orden` (INTEGER). Contiene llaves foráneas a `Clientes` y opcionalmente a `Cupones`.

Consideraciones

El atributo `estado` puede tomar los valores: *pendiente*, *pagada*, *en_despacho*, *entregada* y *cancelada*. El atributo `total` se guarda directamente para evitar recalcular la suma de subtotales de `DetalleOrden` en cada consulta. La FK a `Cupones` puede ser nula porque usar un cupón es opcional.

2.3.6. Entidad DetalleOrden

Especificaciones

Entidad débil de `Ordenes`. Representa cada línea de producto dentro de una orden. Su llave primaria compuesta es `(id_orden, id_producto)`.

Consideraciones

Al ser entidad débil, no puede existir un detalle de orden sin la orden que lo contiene. Si una orden es eliminada, sus detalles se eliminan en cascada. El atributo `precio_unitario` guarda el precio del producto en el momento de la compra, separado del precio actual en `Productos`. El atributo `subtotal` es redundante (`cantidad * precio_unitario`) pero se guarda para hacer más rápidas las consultas de reporte.

2.3.7. Entidad Pagos

Especificaciones

Almacena el registro del pago asociado a cada orden. La llave primaria es `id_pago` (INTEGER). Contiene una llave foránea a `Ordenes`.

Consideraciones

La relación con `Ordenes` es 1:1. Se modeló `Pagos` como entidad separada en lugar de atributos dentro de `Ordenes` para organizar mejor la información. La lógica de pago (método, estado y monto) es independiente de la lógica de la orden. El atributo `metodo` puede ser *tarjeta*, *yape*, *plin*, *efectivo* u otros. El atributo `codigo_autorizacion` almacena el código emitido por la pasarela bancaria al aprobar la transacción, indispensable para auditorías y reclamaciones.

2.3.8. Entidad Envios

Especificaciones

Almacena la información logística de despacho de cada orden. La llave primaria es `id_envio` (INTEGER). Contiene una llave foránea a `Ordenes`.

Consideraciones

Al igual que `Pagos`, se modeló como entidad separada de `Ordenes` para organizar mejor la información. La `direccion_destino` se almacena en `Envios` y no en `Clientes` porque cada orden puede enviarse a una dirección diferente. Las fechas `fecha_despacho` y `fecha_entrega` pueden ser nulas mientras el envío no haya llegado a ese estado. Los atributos `courier` y `numero_seguimiento` permiten al cliente rastrear su pedido en tiempo real a través de la empresa logística asignada, funcionalidad mencionada explícitamente en los requisitos.

2.3.9. Entidad Resenas

Especificaciones

Almacena las valoraciones que los clientes escriben sobre los productos que han comprado. La llave primaria es `id_resena` (INTEGER). Contiene una llave foránea simple a `Clientes` y una llave foránea compuesta dirigida a `DetalleOrden` (`id_orden`, `id_producto`).

Consideraciones

La combinación `(id_orden, id_producto)` debe ser única para asegurar que el cliente emita una sola reseña por cada producto adquirido en una orden específica. El atributo `puntuacion` está limitado al rango 1–5. El atributo `comentario` puede ser nulo para permitir valoraciones solo numéricas sin texto.

2.3.10. Entidad Cupones

Especificaciones

Almacena los códigos de descuento disponibles en la plataforma. La llave primaria es `id_cupon` (INTEGER). El atributo `codigo` es una llave candidata (único y no nulo).

Consideraciones

El atributo `tipo_descuento` indica si el valor de `descuento` es un porcentaje (*porcentaje*) o un monto fijo en soles (*monto_fijo*). Se lleva un conteo `usos_actuales` que aumenta cada vez que se usa el cupón, permitiendo su uso solo mientras sea menor que `usos_maximos`.

3. Modelo Relacional

3.1. Modelo Relacional

1. **Clientes**(id_cliente:INTEGER, nombre:VARCHAR(100), apellido:VARCHAR(100), email:VARCHAR(150), telefono:VARCHAR(20), fecha_registro:DATE, dni:CHAR(8))
2. **Categorias**(id_categoria:INTEGER, nombre:VARCHAR(80), descripcion:VARCHAR(200))
3. **Vendedores**(id_vendedor:INTEGER, nombre_tienda:VARCHAR(150), email:VARCHAR(150), ruc:CHAR(11), calificacion:DECIMAL(3,2))

4. **Productos**(id_producto:INTEGER, nombre:VARCHAR(200), descripcion:VARCHAR(500), precio:DECIMAL(10,2), stock:INTEGER, marca:VARCHAR(100), tipo:VARCHAR(30), *Categorias.id_categoria*:INTEGER, *Vendedores.id_vendedor*:INTEGER)

5. **Cupones**(id_cupon:INTEGER, codigo:VARCHAR(30), descuento:DECIMAL(5,2), tipo_descuento:VARCHAR(20), fecha_vencimiento:DATE, usos_maximos:INTEGER, usos_actuales:INTEGER)

6. **Ordenes**(id_orden:INTEGER, fecha_orden:TIMESTAMP, estado:VARCHAR(30), total:DECIMAL(10,2), *Clientes.id_cliente*:INTEGER, *Cupones.id_cupon*:INTEGER)

7. **DetalleOrden**(Ordenes.id_orden:INTEGER, Productos.id_producto:INTEGER, cantidad:INTEGER, precio_unitario:DECIMAL(10,2), subtotal:DECIMAL(10,2))

8. **Pagos**(id_pago:INTEGER, metodo:VARCHAR(50), estado:VARCHAR(30), fecha_pago:TIMESTAMP, monto:DECIMAL(10,2), codigo_autorizacion:VARCHAR(50), *Ordenes.id_orden*:INTEGER)

9. **Envios**(id_envio:INTEGER, direccion_destino:VARCHAR(300), estado_envio:VARCHAR(30), fecha_despacho:DATE, fecha_entrega:DATE, courier:VARCHAR(100), numero_seguimiento:VARCHAR(50), *Ordenes.id_orden*:INTEGER)

10. **Resenas**(id_resena: INTEGER, puntuacion: INTEGER, comentario: VARCHAR, fecha: DATE, id_cliente: INTEGER, id_orden: INTEGER, id_producto: INTEGER)
    Foreign Key: `id_cliente` → **Clientes**(`id_cliente`)
    Foreign Key Compuesta: `(id_orden, id_producto)` → **DetalleOrden**(`id_orden, id_producto`)
    Restricción: `UNIQUE (id_orden, id_producto)`

3.2. Especificaciones de transformación

3.2.1. Entidades regulares

Llave Primaria	id_cliente
Llave Foránea	—

Tabla 4: Tabla Clientes

Llave Primaria	id_categoria
Llave Foránea	—

Tabla 5: Tabla Categorias

Llave Primaria	id_vendedor
Llave Foránea	—

Tabla 6: Tabla Vendedores

Llave Primaria	id_producto
Llave Foránea	Categorias.id_categoria, Vendedores.id_vendedor

Tabla 7: Tabla Productos

Llave Primaria	id_cupon
Llave Foránea	—

Tabla 8: Tabla Cupones

Llave Primaria	id_orden
Llave Foránea	Clientes.id_cliente, Cupones.id_cupon

Tabla 9: Tabla Ordenes

Llave Primaria	id_pago
Llave Foránea	Ordenes.id_orden

Tabla 10: Tabla Pagos

Llave Primaria	id_resena
Llave Foránea	Clientes.id_cliente, (Ordenes.id_orden, Productos.id_producto) → DetalleOrden

Tabla 11: Tabla Resenas

3.2.2. Entidades débiles

Entidad Débil	DetalleOrden
Entidad Fuerte	Ordenes
Llave Primaria	(Ordenes.id_orden, Productos.id_producto)
Llave Foránea	Ordenes.id_orden, Productos.id_producto

Tabla 12: Tabla DetalleOrden

3.2.3. Entidades superclase/subclase

El modelo no contempla una jerarquía superclase/subclase. Se evaluó modelar `Productos` con subclases `ProductoFisico` y `ProductoDigital`, pero dado que Falabella opera principalmente con productos físicos y que los atributos diferenciadores son escasos, se optó por

incorporar un atributo `tipo` dentro de `Productos` para distinguir ambas categorías sin añadir complejidad innecesaria al modelo.

3.2.4. Relaciones binarias

Entidades	Clientes, Ordenes
Nombre relación	Realiza
Cardinalidad	1:N
Llave Primaria	id_orden
FK en	Ordenes.id_cliente → Clientes

Tabla 13: Relación Realiza (Clientes – Ordenes)

Entidades	Ordenes, Pagos
Nombre relación	Genera_Pago
Cardinalidad	1:1
Llave Primaria	id_pago
FK en	Pagos.id_orden → Ordenes

Tabla 14: Relación Genera_Pago (Ordenes – Pagos)

Entidades	Ordenes, Envios
Nombre relación	Genera_Envio
Cardinalidad	1:1
Llave Primaria	id_envio
FK en	Envios.id_orden → Ordenes

Tabla 15: Relación Genera_Envio (Ordenes – Envios)

Entidades	Productos, Categorias
Nombre relación	Pertenece
Cardinalidad	N:1
Llave Primaria	id_producto
FK en	Productos.id_categoria → Categorias

Tabla 16: Relación Pertenece (Productos – Categorias)

Entidades	Vendedores, Productos
Nombre relación	Ofrece
Cardinalidad	1:N
Llave Primaria	id_producto
FK en	Productos.id_vendedor → Vendedores

Tabla 17: Relación Ofrece (Vendedores – Productos)

Entidades	Clientes, Resenas, DetalleOrden
Nombre relación	Escribe / Califica
Cardinalidad	Resenas 1:1 Escribe:Clientes (cada reseña pertenece a exactamente un cliente); Resenas 1:1 Califica:DetalleOrden
Llave Primaria	id_resena
FK en	Resenas.id_cliente → Clientes; (Resenas.id_orden, Resenas.id_producto) → DetalleOrden

Tabla 18: Relaciones Escribe y Califica (Clientes – Resenas – DetalleOrden)

Entidades	Ordenes, Cupones
Nombre relación	Aplica
Cardinalidad	N:1 (participación parcial en ambos lados)
Llave Primaria	id_orden
FK en	Ordenes.id_cupon → Cupones (nullable)

Tabla 19: Relación Aplica (Ordenes – Cupones)

3.2.5. Relaciones ternarias

El modelo no contempla relaciones ternarias. La relación entre `Clientes`, `Resenas` y `Productos` fue descompuesta en dos relaciones binarias (*Escribe* y *Califica*) absorbidas por la entidad `Resenas`, siguiendo las pautas de transformación del modelo E/R al relacional.

3.3. Diccionario de datos

Nombre Campo	Tipo	PK	FK	Descripción
ID_CLIENTE	INTEGER	X		Identificador numérico único del cliente
NOMBRE	VARCHAR(100)			Nombre(s) del cliente
APELLIDO	VARCHAR(100)			Apellido(s) del cliente
EMAIL	VARCHAR(150)			Correo electrónico único del cliente
TELEFONO	VARCHAR(20)			Número de teléfono de contacto
FECHA_REGISTRO	TIMESTAMP			Fecha en que el cliente creó su cuenta
DNI	CHAR(8)			Documento de identidad único del cliente

Tabla 20: Diccionario – Tabla Clientes

Nombre Campo	Tipo	PK	FK	Descripción
ID_CATEGORIA	INTEGER	X		Identificador numérico único de la categoría
NOMBRE	VARCHAR(80)			Nombre de la categoría (ej. Electrónicos)
DESCRIPCION	VARCHAR(200)			Descripción opcional de la categoría

Tabla 21: Diccionario – Tabla Categorias

Nombre Campo	Tipo	PK	FK	Descripción
ID_VENDEDOR	INTEGER	X		Identificador numérico único del vendedor
NOMBRE_TIENDA	VARCHAR(150)			Nombre comercial de la tienda o vendedor
EMAIL	VARCHAR(150)			Correo electrónico de contacto del vendedor
RUC	CHAR(11)			RUC único del vendedor en Perú
CALIFICACION	DECIMAL(3,2)			Promedio de calificación entre 1.00 y 5.00

Tabla 22: Diccionario – Tabla Vendedores

Nombre Campo	Tipo	PK	FK	Descripción
ID_PRODUCTO	INTEGER	X		Identificador numérico único del producto
NOMBRE	VARCHAR(200)			Nombre del producto en el catálogo
DESCRIPCION	VARCHAR(500)			Descripción detallada del producto
PRECIO	DECIMAL(10,2)			Precio de venta vigente en soles
STOCK	INTEGER			Unidades disponibles en inventario
MARCA	VARCHAR(100)			Marca o fabricante del producto
TIPO	VARCHAR(30)			Indica si el producto es físico o digital
ID_CATEGORIA	INTEGER		X	Categoría a la que pertenece el producto
ID_VENDEDOR	INTEGER		X	Vendedor que ofrece el producto

Tabla 23: Diccionario – Tabla Productos

Nombre Campo	Tipo	PK	FK	Descripción
ID_CUPON	INTEGER	X		Identificador numérico único del cupón
CODIGO	VARCHAR(30)			Código alfanumérico que ingresa el cliente
DESCUENTO	DECIMAL(5,2)			Valor del descuento (porcentaje o monto fijo)
TIPO_DESCUENTO	VARCHAR(20)			Indica si el descuento es porcentual o fijo
FECHA_VENCIMIENTO	DATE			Fecha límite para usar el cupón
USOS_MAXIMOS	INTEGER			Número máximo de veces que puede usarse
USOS_ACTUALES	INTEGER			Número de veces que ya fue utilizado

Tabla 24: Diccionario – Tabla Cupones

Nombre Campo	Tipo	PK	FK	Descripción
ID_ORDEN	INTEGER	X		Identificador numérico único de la orden
FECHA_ORDEN	TIMESTAMP			Fecha y hora en que se creó la orden
ESTADO	VARCHAR(30)			Estado actual de la orden. Valores posibles: 'Procesado', 'Enviado', 'Devuelto'
TOTAL	DECIMAL(10,2)			Monto total de la orden en soles
ID_CLIENTE	INTEGER		X	Cliente que realizó la orden
ID_CUPON	INTEGER		X	Cupón aplicado a la orden (nullable)

Tabla 25: Diccionario – Tabla Ordenes

Nombre Campo	Tipo	PK	FK	Descripción
ID_ORDEN	INTEGER	X	X	Orden a la que pertenece el detalle
ID_PRODUCTO	INTEGER	X	X	Producto incluido en la orden
CANTIDAD	INTEGER			Unidades del producto solicitadas
PRECIO_UNITARIO	DECIMAL(10,2)			Precio del producto al momento de la compra
SUBTOTAL	DECIMAL(10,2)			Resultado de cantidad por precio unitario

Tabla 26: Diccionario – Tabla DetalleOrden

Nombre Campo	Tipo	PK	FK	Descripción
ID_PAGO	INTEGER	X		Identificador numérico único del pago
METODO	VARCHAR(50)			Método de pago utilizado
ESTADO	VARCHAR(30)			Estado del pago (aprobado, rechazado, etc.)
FECHA_PAGO	TIMESTAMP			Fecha y hora en que se procesó el pago
MONTO	DECIMAL(10,2)			Monto cobrado en soles
CODIGO_AUTORIZACION	VARCHAR(50)			Código emitido por la pasarela al aprobar
ID_ORDEN	INTEGER		X (UNIQUE)	Orden a la que corresponde este pago

Tabla 27: Diccionario – Tabla Pagos

Nombre Campo	Tipo	PK	FK	Descripción
ID_ENVIO	INTEGER	X		Identificador numérico único del envío
DIRECCION_DESTINO	VARCHAR(300)			Dirección completa de entrega
ESTADO_ENVIO	VARCHAR(30)			Estado del envío
FECHA_DESPACHO	TIMESTAMP			Fecha en que salió del almacén (nullable)
FECHA_ENTREGA	TIMESTAMP			Fecha en que fue entregado (nullable)
COURIER	VARCHAR(100)			Empresa de transporte asignada al envío
NUMERO_SEGUIMIENTO	VARCHAR(50)			Código de rastreo del paquete
ID_ORDEN	INTEGER		X (UNIQUE)	Orden a la que corresponde este envío

Tabla 28: Diccionario – Tabla Envios

4. Implementación de la base de datos

4.1. Creación de Tablas en PostgreSQL

La estructura física de la base de datos fue implementada en PostgreSQL 18.3 a partir del modelo relacional descrito en la Sección 3. El script completo de creación (`schema.sql`) define las 10 tablas con sus respectivas restricciones de integridad: llaves primarias (`SERIAL PRIMARY KEY`), llaves foráneas (`REFERENCES`), restricciones de dominio (`CHECK`) y restricciones de unicidad (`UNIQUE`).

A modo de ejemplo, se muestra la definición de la tabla `Productos`, que ilustra el uso de

Nombre Campo	Tipo	PK	FK	Descripción
ID_RESENA	INTEGER	X		Identificador numérico único de la reseña (SERIAL).
PUNTUACION	INTEGER			Valoración del 1 al 5 otorgada al producto (CHECK).
COMENTARIO	VARCHAR(1000)			Texto opcional con los detalles de la reseña.
FECHA	DATE			Fecha de publicación (DEFAULT CURRENT_DATE).
ID_CLIENTE	INTEGER		X	Cliente que escribió la reseña.
ID_ORDEN	INTEGER		X	Orden de la compra. Junto con ID_PRODUCTO es UNIQUE y FK compuesta.
ID_PRODUCTO	INTEGER		X	Producto calificado. Parte de la FK compuesta hacia DetalleOrden.

Tabla 29: Diccionario – Tabla Resenas

restricciones de dominio para impedir valores inválidos (precios negativos, stock negativo, tipos fuera de catálogo):

```
CREATE TABLE Clientes (
    id_cliente SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    telefono VARCHAR(20),
    fecha_registro TIMESTAMP NOT NULL DEFAULT NOW(),
    dni CHAR(8) NOT NULL UNIQUE
);

CREATE TABLE Categorias (
    id_categoria SERIAL PRIMARY KEY,
    nombre VARCHAR(80) NOT NULL,
    descripcion VARCHAR(200)
);

CREATE TABLE Vendedores (
    id_vendedor SERIAL PRIMARY KEY,
    nombre_tienda VARCHAR(150) NOT NULL,
    email VARCHAR(150),
    ruc CHAR(11) NOT NULL UNIQUE,
    calificacion DECIMAL(3,2) DEFAULT 0 CHECK (calificacion BETWEEN 0 AND 5)
);

CREATE TABLE Productos (
```

```
    id_producto SERIAL PRIMARY KEY,
    nombre VARCHAR(200) NOT NULL,
    descripcion VARCHAR(500),
    precio DECIMAL(10,2) NOT NULL CHECK (precio > 0),
    stock INTEGER NOT NULL CHECK (stock >= 0),
    marca VARCHAR(100),
    tipo VARCHAR(30) CHECK (tipo IN ('fisico','digital')),
    id_categoria INTEGER NOT NULL REFERENCES Categorias(id_categoria),
    id_vendedor INTEGER NOT NULL REFERENCES Vendedores(id_vendedor)
);

CREATE TABLE Cupones (
    id_cupon SERIAL PRIMARY KEY,
    codigo VARCHAR(30) NOT NULL UNIQUE,
    descuento DECIMAL(5,2) NOT NULL CHECK (descuento > 0),
    tipo_descuento VARCHAR(20) NOT NULL CHECK (tipo_descuento IN ('porcentaje','monto_fi
    fecha_vencimiento DATE NOT NULL,
    usos_maximos INTEGER NOT NULL CHECK (usos_maximos > 0),
    usos_actuales INTEGER NOT NULL DEFAULT 0 CHECK (usos_actuales >= 0)
);

CREATE TABLE Ordenes (
    id_orden SERIAL PRIMARY KEY,
    fecha_orden TIMESTAMP NOT NULL DEFAULT NOW(),
    estado VARCHAR(30) NOT NULL CHECK (estado IN ('pendiente','pagada','en_despacho','en
    total DECIMAL(10,2) NOT NULL DEFAULT 0,
    id_cliente INTEGER NOT NULL REFERENCES Clientes(id_cliente),
    id_cupon INTEGER REFERENCES Cupones(id_cupon)
);

CREATE TABLE DetalleOrden (
    id_orden INTEGER NOT NULL REFERENCES Ordenes(id_orden) ON DELETE CASCADE,
    id_producto INTEGER NOT NULL REFERENCES Productos(id_producto),
    cantidad INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unitario DECIMAL(10,2) NOT NULL CHECK (precio_unitario > 0),
    subtotal DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id_orden, id_producto)
);

CREATE TABLE Pagos (
    id_pago SERIAL PRIMARY KEY,
    metodo VARCHAR(50) NOT NULL CHECK (metodo IN ('tarjeta','yape','plin','efectivo')),
    estado VARCHAR(30) NOT NULL CHECK (estado IN ('aprobado','rechazado','pendiente')),
    fecha_pago TIMESTAMP NOT NULL DEFAULT NOW(),
    monto DECIMAL(10,2) NOT NULL CHECK (monto > 0),
```

```
    codigo_autorizacion VARCHAR(50),
    id_orden INTEGER NOT NULL UNIQUE REFERENCES Ordenes(id_orden)
);

CREATE TABLE Envios (
    id_envio SERIAL PRIMARY KEY,
    direccion_destino VARCHAR(300) NOT NULL,
    estado_envio VARCHAR(30) NOT NULL CHECK (estado_envio IN ('preparando','despachado',
    fecha_despacho TIMESTAMP,
    fecha_entrega TIMESTAMP,
    courier VARCHAR(100),
    numero_seguimiento VARCHAR(50),
    id_orden INTEGER NOT NULL UNIQUE REFERENCES Ordenes(id_orden)
);

CREATE TABLE Resenas (
    id_resena SERIAL PRIMARY KEY,
    puntuacion INTEGER NOT NULL CHECK (puntuacion BETWEEN 1 AND 5),
    comentario VARCHAR(1000),
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    id_cliente INTEGER NOT NULL REFERENCES Clientes(id_cliente),
    id_orden INTEGER NOT NULL,
    id_producto INTEGER NOT NULL,
    UNIQUE (id_orden, id_producto),
    FOREIGN KEY (id_orden, id_producto) REFERENCES DetalleOrden(id_orden, id_producto)
);

--Calcular el subtotal automáticamente
CREATE OR REPLACE FUNCTION fn_subtotal() RETURNS TRIGGER AS $$
BEGIN
    NEW.subtotal := NEW.cantidad * NEW.precio_unitario;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_subtotal
    BEFORE INSERT OR UPDATE ON DetalleOrden
    FOR EACH ROW EXECUTE FUNCTION fn_subtotal();

--Recalcular el total de la orden
CREATE OR REPLACE FUNCTION fn_total_orden() RETURNS TRIGGER AS $$
BEGIN
    UPDATE Ordenes
    SET total = (SELECT COALESCE(SUM(subtotal),0) FROM DetalleOrden WHERE id_orden = NEW
    WHERE id_orden = NEW.id_orden;
    RETURN NEW;
```

```
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_total_orden
    AFTER INSERT OR UPDATE ON DetalleOrden
    FOR EACH ROW EXECUTE FUNCTION fn_total_orden();

--Descuento de stock al confirmarse el pago (estado pasa a verse como 'pagada')
CREATE OR REPLACE FUNCTION fn_descontar_stock() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.estado = 'pagada' AND OLD.estado IS DISTINCT FROM 'pagada' THEN
        UPDATE Productos p
        SET stock = stock - d.cantidad
        FROM DetalleOrden d
        WHERE d.id_orden = NEW.id_orden AND p.id_producto = d.id_producto;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_descontar_stock
    AFTER UPDATE ON Ordenes
    FOR EACH ROW EXECUTE FUNCTION fn_descontar_stock();

--Incrementador de usos_actuales del cupon cuando creas la orden
CREATE OR REPLACE FUNCTION fn_usar_cupon() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.id_cupon IS NOT NULL THEN
        UPDATE Cupones SET usos_actuales = usos_actuales + 1
        WHERE id_cupon = NEW.id_cupon AND usos_actuales < usos_maximos;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_usar_cupon
    AFTER INSERT ON Ordenes
    FOR EACH ROW EXECUTE FUNCTION fn_usar_cupon();
```

Adicionalmente, se implementaron 4 *triggers* en pgSQL para mantener la consistencia transaccional sin depender de la lógica de aplicación:

- `trg_subtotal`: calcula automáticamente el `subtotal` de cada línea de `DetalleOrden` como `cantidad * precio_unitario`.
- `trg_total_orden`: recalcula el campo `total` de `Ordenes` cada vez que se inserta o actualiza una línea de `DetalleOrden`.
- `trg_descontar_stock`: descuenta automáticamente el `stock` de `Productos` únicamente cuando el estado de una orden cambia a *'pagada'*, aplicando la regla semántica #9 del

modelo E/R.

- `trg_usar_cupon`: incrementa `usos_actuales` en `Cupones` al crearse una orden que referencia dicho cupón, validando que no se exceda `usos_maximos`.

El script fue ejecutado de forma idéntica en cuatro bases de datos independientes (`falabella_1k`, `falabella_10k`, `falabella_100k`, `falabella_1m`), una por cada escenario de volumen de datos definido para la experimentación.

4.2. Carga de datos

La carga de datos se realizó mediante un script en Python (con ayuda de la IA Claude ya que no teníamos experiencia manejando la librería Faker) (`generar_datos.py`) que utiliza la librería **Faker** para generar datos sintéticos realistas y **psycopg2** para insertarlos directamente en PostgreSQL mediante inserciones por lotes (*batch insert* con `execute_values`), lo que permite cargar grandes volúmenes de datos de forma eficiente.

El script respeta el orden de dependencias entre tablas (entidades sin llaves foráneas primero, entidades dependientes después):

$$\text{Categorias} \rightarrow \text{Vendedores} \rightarrow \text{Clientes} \rightarrow \text{Cupones} \rightarrow \text{Productos} \rightarrow \text{Ordenes} \rightarrow \text{DetalleOrden} \rightarrow \text{Pagos} \rightarrow \text{Envios} \rightarrow \text{Resenas}$$

Se definieron cuatro escenarios de carga con cardinalidades crecientes, de modo que cada tabla escala de forma proporcional:

Tabla	**1K**	**10K**	**100K**	**1M**
Clientes	200	2,000	20,000	200,000
Vendedores	30	100	500	2,000
Categorías	8	8	8	8
Cupones	50	200	1,000	5,000
Productos	300	3,000	30,000	300,000
Ordenes	1,000	10,000	100,000	1,000,000

Tabla 30: Cardinalidades planificadas por escenario

Las tablas dependientes de `Ordenes` (`DetalleOrden`, `Pagos`, `Envios`, `Resenas`) se generaron de forma condicionada al estado de cada orden: solo las órdenes con estado distinto de *'pendiente'* generan un registro en `Pagos`, y solo las que están *'en_despacho'* o *'entregada'* generan un registro en `Envios`, simulando el flujo real de un pedido de e-commerce. Los conteos finales obtenidos tras la carga confirman esta proporcionalidad:

Tabla	1K	10K	100K	1M
Ordenes	1,000	10,000	100,000	1,000,000
DetalleOrden	3,004	29,844	300,108	2,999,435
Pagos	957	9,506	95,042	949,828
Envios	763	7,607	75,037	749,536
Resenas	901	8,953	90,032	899,830

Tabla 31: Registros efectivamente cargados por escenario

Tras cada carga se exportó la base correspondiente mediante `pg_dump`, generando cuatro archivos de respaldo (`dump_1k.sql`, `dump_10k.sql`, `dump_100k.sql`, `dump_1m.sql`) que permitieron distribuir un único conjunto de datos consistente entre los integrantes del equipo para garantizar que las pruebas de optimización se realizaran sobre exactamente la misma información.

4.3. Simulación de Datos Faltantes

Dado que no se encontró un dataset público de comercio electrónico peruano con el nivel de detalle transaccional requerido por el modelo (órdenes, pagos, envíos y reseñas enlazadas entre sí), se optó por generar el 100 % de los datos de forma sintética, asegurando coherencia referencial completa entre tablas. Para aproximar el comportamiento de un sistema real, se simularon de forma intencional los siguientes casos de datos faltantes o incompletos:

- **Comentarios de reseñas nulos**: aproximadamente un 30 % de las reseñas generadas no incluyen texto en `comentario`, simulando valoraciones puramente numéricas.
- **Fechas de envío incompletas**: `fecha_despacho` y `fecha_entrega` permanecen nulas mientras la orden no alcanza el estado correspondiente, replicando el comportamiento real de un pedido en tránsito.
- **Cupón opcional**: solo un 25 % de las órdenes generadas tienen un `id_cupon` asociado; el resto queda en `NULL`, reflejando que el uso de cupones es opcional en el proceso de compra.
- **Quiebres de stock**: se forzó que aproximadamente un 8 % de los productos generados tuvieran `stock = 0`, para poder representar de forma realista los escenarios de quiebre de inventario mencionados en los requisitos (Sección 1.8.1) y necesarios para las consultas de optimización (ver Sección 5.1).

Estas decisiones de diseño permiten que las consultas de agregación y filtrado del experimento (Sección 5) trabajen sobre datos con la misma variabilidad e incompletitud que existiría en una base de datos productiva real, en lugar de un conjunto de datos artificialmente perfecto.

5. Optimización y Experimentación

5.1. Consultas SQL para el experimento

5.1.1. Descripción del tipo de consultas seleccionadas

La **Consulta 1** fue diseñada para el equipo de Marketing, Fidelización y Experiencia del Cliente de Falabella Perú, con el objetivo de identificar y mitigar el riesgo de fuga de los compradores de alto valor o "Clientes VIP" (definidos como aquellos cuyas órdenes de compra superan el doble del promedio general de transacciones en la plataforma). Esta consulta cruza el comportamiento financiero de estos usuarios Premium con su nivel de insatisfacción post-venta, rastreando específicamente a aquellos que han dejado evaluaciones críticas o bajas calificaciones en sus compras recientes. El propósito de negocio es activar de manera proactiva campañas de retención urgentes, implementar estrategias de compensación (como cupones de descuento exclusivos o atención al cliente prioritaria) y monitorear la calidad del servicio logístico en el segmento corporativo de mayor valor para el Marketplace. La consulta involucra 4 tablas (`Clientes`, `Ordenes`, `DetalleOrden`, `Resenas`), una subconsulta para calcular dinámicamente el umbral de facturación VIP, múltiples funciones de agregación (`COUNT DISTINCT`, `SUM`, `AVG`), ordenamiento descendente por volumen de inversión y una cláusula `HAVING` encargada de aislar únicamente a los clientes premium cuya calificación promedio histórica sea igual o inferior a 2.5 estrellas.

La **Consulta 2** fue diseñada para el equipo comercial y de Marketplace de Falabella Perú, con el objetivo de identificar y evaluar el rendimiento de los vendedores externos que comercializan productos súper premium o exclusivos (definidos como aquellos cuyo precio de lista supera el doble del promedio general de todo el catálogo), cruzando esta información financiera con la satisfacción del cliente reflejada en sus calificaciones. El propósito de negocio es medir la rentabilidad real de estas tiendas basándose en el volumen de ingresos generados por sus artículos más caros, permitiendo a la plataforma destacar a los mejores vendedores, monitorear la calidad de estos productos de élite y optimizar las estrategias de cobro de comisiones. La consulta involucra 4 tablas (Vendedores, Productos, DetalleOrden, Resenas), una subconsulta para calcular dinámicamente este umbral de precio, múltiples funciones de agregación (COUNT DISTINCT, SUM, AVG) y una cláusula HAVING para el filtro de tiendas que hayan concretado al menos una venta de este tipo.

La **Consulta 3** fue diseñada para el equipo de finanzas y gestión de alianzas del *Marketplace* de Falabella Perú, con el objetivo de identificar a los 5 proveedores externos que han generado el mayor volumen de ingresos monetarios reales (basado exclusivamente en órdenes con estado `.entregada"`, que representa el ciclo de vida transaccional completamente finalizado: pago procesado, despacho ejecutado y entrega confirmada por el courier) durante los últimos 30 días, contrastando este rendimiento financiero con la satisfacción del cliente mediante la calificación promedio de sus productos. El propósito de negocio es doble: por un lado, liquidar con precisión y rapidez las comisiones mensuales sobre ventas efectivas y, por otro, retroalimentar el motor de recomendaciones de la plataforma para otorgar mayor visibilidad orgánica a los vendedores que no solo venden más, sino que mantienen altos estándares de calidad (buenas reseñas). A nivel técnico, esta consulta representa una carga analítica pesada, ya que involucra 5 tablas de alto crecimiento (`Vendedores`, `Productos`, `DetalleOrden`,

`Ordenes`, `Resenas`), múltiples `INNER JOIN`s sumados a un `LEFT JOIN` (para no descartar ventas de productos que aún no han sido reseñados), filtrado por cálculo de intervalos de fechas (`INTERVAL`), y la ejecución simultánea de múltiples funciones de agregación (`SUM`, `COUNT DISTINCT`, `AVG`) sobre millones de tuplas agrupadas.

La **Consulta 4** fue diseñada para el equipo de marketing de Falabella Perú, con el objetivo de identificar cupones promocionales que están próximos a vencer (en los siguientes 30 días) y que presentan una tasa de uso baja (menos del 50 % de sus usos máximos), cruzando esta información con la categoría de productos que actualmente concentra más quiebres de stock. El propósito de negocio es evitar que el equipo de marketing refuerce campañas promocionales sobre productos que no podrán entregarse por falta de inventario, optimizando así el gasto en descuentos. La consulta involucra 4 tablas (`Cupones`, `Ordenes`, `DetalleOrden`, `Productos`, `Categorias`), una subconsulta correlacionada para determinar la categoría con más quiebres de stock, funciones de agregación (`COUNT DISTINCT`) y una cláusula `HAVING` para el filtro de baja tasa de uso.

5.1.2. Implementación de consultas en SQL

Consulta 1:

```
EXPLAIN ANALYZE
SELECT
    c.id_cliente,
    c.nombre AS nombre_cliente,
    c.email,
    COUNT(DISTINCT o.id_orden) AS total_ordenes_realizadas,
    SUM(d.subtotal) AS monto_total_invertido,
    ROUND(AVG(r.puntuacion), 2) AS peor_puntuacion_otorgada
FROM Clientes c
JOIN Ordenes o ON c.id_cliente = o.id_cliente
JOIN DetalleOrden d ON o.id_orden = d.id_orden
JOIN Resenas r ON d.id_producto = r.id_producto
WHERE d.subtotal > (
    SELECT AVG(subtotal) * 2
    FROM DetalleOrden
)
GROUP BY c.id_cliente, c.nombre, c.email
HAVING AVG(r.puntuacion) <= 2.5
ORDER BY monto_total_invertido DESC;
```

Consulta 2:

```
EXPLAIN ANALYZE
SELECT
    v.nombre_tienda,
    COUNT(DISTINCT d.id_orden) AS total_pedidos_premium,
    SUM(d.subtotal) AS ingresos_totales,
```

```
    ROUND(AVG(r.puntuacion), 2) AS reputacion_promedio
FROM Vendedores v
JOIN Productos p ON v.id_vendedor = p.id_vendedor
JOIN DetalleOrden d ON p.id_producto = d.id_producto
JOIN Resenas r ON p.id_producto = r.id_producto
WHERE p.precio > (
    SELECT AVG(precio) * 2
    FROM Productos
)
GROUP BY v.id_vendedor, v.nombre_tienda
HAVING COUNT(DISTINCT d.id_orden) >= 1
ORDER BY ingresos_totales DESC;
```

Consulta 3:

```
SELECT
    v.id_vendedor,
    v.nombre_tienda,
    v.email,
    v.ruc,
    SUM(d.subtotal)AS ingresos_totales,
    COUNT(DISTINCT o.id_orden)AS total_ordenes_completadas,
    COUNT(DISTINCT p.id_producto)AS productos_distintos_vendidos,
    ROUND(AVG(r.puntuacion), 2)AS calificacion_promedio,
    COUNT(r.id_resena)AS total_resenas
FROM vendedores v

INNER JOIN productos p
    ON p.id_vendedor = v.id_vendedor

INNER JOIN detalleorden d
    ON d.id_producto = p.id_producto

INNER JOIN ordenes o
    ON o.id_orden = d.id_orden
   AND o.estado = 'entregada'
   AND o.fecha_orden >= NOW() - INTERVAL '30 days'

LEFT JOIN resenas r
    ON r.id_producto = p.id_producto
   AND r.id_orden = o.id_orden

GROUP BY
    v.id_vendedor,
    v.nombre_tienda,
```

```
    v.email,
    v.ruc

ORDER BY
    ingresos_totales DESC

LIMIT 5;
```

Consulta 4:

```
SELECT c.codigo, c.descuento, c.tipo_descuento, c.fecha_vencimiento,
       c.usos_actuales, c.usos_maximos,
       cat.nombre AS categoria_mas_afectada,
       COUNT(DISTINCT p.id_producto) AS productos_sin_stock
FROM Cupones c
JOIN Ordenes o ON o.id_cupon = c.id_cupon
JOIN DetalleOrden d ON d.id_orden = o.id_orden
JOIN Productos p ON p.id_producto = d.id_producto
JOIN Categorias cat ON cat.id_categoria = p.id_categoria
WHERE c.fecha_vencimiento BETWEEN CURRENT_DATE AND CURRENT_DATE + INTERVAL '30 days'
  AND p.id_categoria = (
      SELECT p2.id_categoria
      FROM Productos p2
      WHERE p2.stock = 0
      GROUP BY p2.id_categoria
      ORDER BY COUNT(*) DESC
      LIMIT 1
  )
GROUP BY c.id_cupon, c.codigo, c.descuento, c.tipo_descuento,
         c.fecha_vencimiento, c.usos_actuales, c.usos_maximos, cat.nombre
HAVING c.usos_actuales < (c.usos_maximos * 0.5)
ORDER BY c.fecha_vencimiento ASC;
```

5.2. Metodología del experimento

Para garantizar resultados comparables entre los cuatro escenarios de volumen de datos (1,000, 10,000, 100,000 y 1,000,000 registros), se siguió el mismo protocolo de medición para las 4 consultas del experimento:

1. Se ejecuta `VACUUM;` sobre la base de datos correspondiente antes de cada bloque de mediciones, para descartar tuplas muertas y actualizar las estadísticas internas que utiliza el optimizador de PostgreSQL.
2. Se ejecuta la consulta mediante `EXPLAIN ANALYZE`, lo que permite obtener tanto el plan de ejecución elegido por el optimizador como el tiempo real de ejecución (*Execution Time*).

3. Cada consulta se ejecuta 5 veces consecutivas en el mismo escenario, registrando el *Execution Time* de cada corrida. Se reporta el promedio de las 5 corridas como tiempo representativo, minimizando el efecto de variaciones puntuales (por ejemplo, la primera ejecución suele ser más lenta debido a que los datos aún no están en caché, un efecto conocido como *cold cache*).

4. Este procedimiento se repite primero **sin índices** (utilizando únicamente los índices automáticos generados por las llaves primarias) y luego **con índices** (tras crear los índices específicos de cada consulta), permitiendo aislar el efecto de la optimización.

5. El procedimiento completo (sin índices y con índices, 5 corridas cada uno) se ejecuta de forma idéntica en los 4 escenarios de volumen, sobre bases de datos independientes que contienen exactamente el mismo conjunto de datos sintéticos (ver Sección 4.2), para que las diferencias observadas se atribuyan únicamente al volumen de datos y al uso de índices, y no a variaciones en el contenido.

Las especificaciones del entorno de hardware y software utilizado se detallan en la Sección 5.4.

5.3. Optimización de consultas

5.3.1. Planes de índices para Consulta 1

Para optimizar la Consulta 1 se crearon 3 índices, justificados por las columnas utilizadas en las cláusulas `WHERE` y `JOIN`:

```
CREATE INDEX idx_ordenes_cliente ON Ordenes(id_cliente);
CREATE INDEX idx_detalle_subtotal ON DetalleOrden(subtotal);
CREATE INDEX idx_detalle_orden ON DetalleOrden(id_orden);
```

Plan sin índices (escenario 1,000,000 registros): En este escenario, antes de crear las estructuras de indexación, PostgreSQL se ve obligado a resolver la consulta realizando un escaneo secuencial masivo (*Seq Scan*) sobre la tabla `DetalleOrden` para evaluar la subconsulta aritmética y calcular el promedio global del subtotal. Posteriormente, propaga estos registros hacia los acoplamientos principales mediante un algoritmo de *Hash Join* costoso con las tablas `Clientes`, `Ordenes` y `Resenas`, alcanzando un tiempo de ejecución promedio alto de 3061.90 ms.

Plan con índices (escenario 1,000,000 registros): Tras aplicar las estructuras de indexación, el plan de ejecución confirma que los índices son selectivamente aprovechados por el optimizador de consultas:

- `idx_detalle_subtotal` es explotado mediante un *Index Scan* directo para aislar rápidamente las filas de la tabla `DetalleOrden` cuyo subtotal supera el umbral dinámico calculado por la subconsulta.
- `idx_ordenes_cliente` y `idx_detalle_orden` optimizan la fase de acoplamiento lógicos, transformando los barridos secuenciales en accesos directos por punteros lógicos, agilizando la transferencia de datos entre `Clientes` y `Ordenes`.

A pesar de que las operaciones de emparejamiento mejoraron lógicamente, el tiempo de ejecución promedio global solo se redujo un 4.70 % (de 3061.90 ms a 2918.06 ms). Este comportamiento evidencia que el costo dominante de la consulta no proviene del acceso o filtrado puntual de las filas en las tablas base, sino de la naturaleza analítica de la subconsulta y sus agregaciones en la cláusula `HAVING`.

El plan de ejecución revela que para procesar la subconsulta del promedio se requiere un escaneo masivo de agregación (*Aggregate*) que arrastra el procesamiento aritmético completo de millones de registros en la memoria caché, manteniendo la consulta acotada por el uso intensivo de la CPU. De este modo, se demuestra que aunque un índice optimice la búsqueda física de los datos, no puede reducir de forma autónoma el costo analítico inherente a operaciones matemáticas masivas aplicadas sobre un resultado intermedio genuinamente grande en escenarios a gran escala.

Etapa del plan	**Operación / Índice utilizado**	**Filas (actual)**
Subconsulta (Cálculo del promedio global)	HashAggregate sobre tabla `DetalleOrden` completa	1
Filtro de Subtotales VIP	Index Scan usando `idx_detalle_subtotal`	42,115
Join Clientes–Ordenes	Index Scan usando `idx_ordenes_cliente` (Hash Join)	114,826
Join Ordenes–DetalleOrden	Index Scan usando `idx_detalle_orden`	114,826
Agregación y Filtro por Reseñas	HashAggregate con evaluación de cláusula `HAVING`	1,340

Tabla 32: Resumen del plan de ejecución de la Consulta 2 con índices, escenario de 1,000,000 registros

Métrica del plan	**Primer Intento (Caché Fría)**	**Intentos Post. (Caché Caliente)**
Buffers *shared hit* (RAM)	682,410 bloques	512,054 bloques
Buffers *shared read* (Disco)	84,115 bloques	0 bloques
Planning Time	1.950 ms	0.980 ms
Execution Time (Promedio)	3202.602 ms	2918.062 ms

Tabla 33: Métricas clave del plan de ejecución, Consulta 1 con índices (1M registros)

5.3.2. Planes de índices para Consulta 2

Para optimizar la Consulta 2 se crearon 3 índices, justificados por las columnas críticas utilizadas en los filtros de selección y en las condiciones de acoplamiento (`JOIN`):

```
CREATE INDEX idx_productos_precio ON Productos(precio);
CREATE INDEX idx_detalle_producto ON DetalleOrden(id_producto);
CREATE INDEX idx_resenas_producto ON Resenas(id_producto);
```

Plan sin índices (escenario 1,000,000 registros): En este escenario, antes de crear las estructuras de indexación, PostgreSQL se ve obligado a resolver la consulta mediante un escaneo secuencial masivo (*Seq Scan*) sobre la tabla `Productos` para poder calcular el valor de la subconsulta y evaluar la condición del filtro. Posteriormente, propaga el conjunto resultante hacia los acoplamientos principales usando algoritmos de *Hash Join* con las tablas `DetalleOrden` y `Resenas`, alcanzando un tiempo de ejecución promedio de 228.32 ms.

Plan con índices y análisis de escala (100,000 vs. 1,000,000 registros): Tras aplicar los índices, se observó un comportamiento sumamente interesante que evidencia la naturaleza del optimizador de PostgreSQL. En el escenario de 100,000 registros, el tiempo promedio aumentó de 12.14 ms a 50.85 ms. Este fenómeno se conoce como *Index Overhead*: cuando el volumen de datos es lo suficientemente pequeño, el motor prefiere barrer la tabla directamente en memoria caché; forzar el uso de índices añade pasos adicionales (recorrer el árbol del índice y luego saltar a la tabla), penalizando el rendimiento.

Sin embargo, en el escenario crítico de 1,000,000 de registros, el panorama cambia drásticamente. El tiempo de ejecución promedio se desploma a tan solo 65.59 ms, lo que representa una mejora del 71.3 % con respecto al plan original. A gran escala, el optimizador confirma que los 3 índices son altamente eficientes debido a la selectividad de la regla de negocio implementada:

- `idx_productos_precio` se utiliza mediante un *Index Only Scan* dentro de la subconsulta para determinar el umbral del doble del precio promedio, aislando inmediatamente el segmento de productos súper premium.
- `idx_detalle_producto` y `idx_resenas_producto` transforman los costosos escaneos masivos de los joins en búsquedas indexadas directas (*Index Scan*), procesando únicamente las filas que cumplen con el criterio de exclusividad.

Análisis de Buffers y Estabilización de Caché: Un hallazgo crítico durante las pruebas de carga en el escenario de 1M fue el comportamiento de los buffers del motor entre la primera ejecución (caché fría) y las subsecuentes (caché caliente). En el primer intento, el motor incurrió en 13 lecturas físicas de disco (*shared read*) y un *shared hit* de 119 bloques, elevando el tiempo a 251.74 ms debido al costo del I/O físico. A partir de la segunda ejecución, los datos se estabilizaron por completo en la memoria RAM, eliminando las lecturas de disco (*shared read* igual a 0) y reduciendo el consumo a un *shared hit* constante de 54 bloques, lo que permitió mitigar la latencia y consolidar el tiempo de ejecución en la franja óptima de los 60 ms.

Etapa del plan	Operación / Índice utilizado	Filas (actual)
Subconsulta (Cálculo del precio promedio)	Index Only Scan usando `idx_productos_precio`	1
Filtro de Productos Súper Premium	Index Scan usando `idx_productos_precio`	14,205
Join Productos–DetalleOrden	Index Scan usando `idx_detalle_producto`	28,410
Join final con Resenas	Index Scan usando `idx_resenas_producto`	28,410
Agregación y ordenamiento final	GroupAggregate y Sort en memoria	450

Tabla 34: Resumen del plan de ejecución de la Consulta 2 con índices, escenario de 1,000,000 registros

Métrica del plan	Primer Intento (Caché Fría)	Intentos Post. (Caché Caliente)
Buffers *shared hit* (RAM)	119 bloques	54 bloques
Buffers *shared read* (Disco)	13 bloques	0 bloques
Planning Time	1.850 ms	0.930 ms
Execution Time (Promedio)	251.743 ms	65.371 ms

Tabla 35: Comparativa de métricas clave del plan de ejecución (1M registros con índices)

5.3.3. Planes de índices para Consulta 3

Para optimizar la Consulta 3 se crearon 4 índices, justificados por las columnas críticas utilizadas en las cláusulas de filtrado (`WHERE`) y en las condiciones de acoplamiento (`JOIN`):

```
CREATE INDEX idx_ordenes_estado_fecha ON ordenes(estado, fecha_orden);
CREATE INDEX idx_detalleorden_producto ON detalleorden(id_producto);
CREATE INDEX idx_productos_vendedor ON productos(id_vendedor);
CREATE INDEX idx_resenas_producto_orden ON resenas(id_producto, id_orden);
```

Plan sin índices (escenario 1,000,000 registros): En este escenario, antes de crear las estructuras de indexación, PostgreSQL resuelve la consulta mediante un escaneo secuencial masivo (`Seq Scan`) sobre todas las tablas involucradas. Al no existir estructuras de acceso directo, el motor debe recorrer la totalidad de las tuplas de `Ordenes` para evaluar simultáneamente el filtro de estado (`'entregada'`) y el intervalo de fechas (`INTERVAL '30 days'`), propagando el conjunto resultante hacia los acoplamientos con `Productos`, `DetalleOrden` y `Resenas` mediante algoritmos de `Hash Join` sobre millones de filas, alcanzando un tiempo de ejecución promedio de 1,631.48 ms.

Plan con índices (escenario 1,000,000 registros): Tras aplicar las estructuras de indexación, el plan de ejecución revela un comportamiento crítico del optimizador de consultas de PostgreSQL: solo uno de los cuatro índices creados fue seleccionado para la ejecución, mientras que los tres restantes fueron descartados en favor de escaneos secuenciales paralelos debido a la relación costo-beneficio estimada por el motor:

- El índice `idx_ordenes_estado_fecha` fue explotado exitosamente mediante un *Bitmap Index Scan*. Este mecanismo construyó un mapa de bits en memoria para localizar eficientemente las órdenes que cumplían con el criterio temporal y de estado, evitando el barrido secuencial del millón de registros de la tabla `Ordenes`.
- El índice `idx_productos_vendedor` fue ignorado por el optimizador debido al reducido tamaño de la tabla `Vendedores` (2,000 filas). Para el motor, resulta matemáticamente más económico realizar un *Seq Scan* directo sobre la tabla en memoria que incurrir en el costo de E/S de consultar el archivo del índice.
- Los índices `idx_detalleorden_producto` y `idx_resenas_producto_orden`, a pesar de existir, no fueron utilizados. El volumen masivo de datos intermedios obligó a PostgreSQL a procesar los acoplamientos mediante *Parallel Hash Joins*. Para alimentar de forma óptima las tablas Hash en memoria con cientos de miles de registros, el motor determinó que un *Parallel Seq Scan* multifilo sobre `detalleorden` y `resenas` era sustancialmente más rápido que realizar accesos aleatorios indexados (*Random Disk Access*) para cada fila. Para la tabla `Productos`, el motor prefirió apoyarse en el índice nativo de su llave primaria (`productos_pkey`).

Como consecuencia de este descarte estratégico, la estrategia dominante para procesar el grueso de los datos continuó siendo el barrido secuencial distribuido en hilos paralelos. Esto explica por qué en el escenario de 1,000,000 de registros la mejora promedio obtenida fue de un **8.66 %** (reduciendo el tiempo de 1,631.48 ms a 1,490.20 ms). Este comportamiento demuestra empíricamente que el verdadero cuello de botella de la Consulta 3 no reside en el filtrado inicial, sino en el costo estructural ineludible de los `JOIN`s encadenados y el volumen de filas sobrevivientes que deben ordenarse en memoria (*top-N heapsort*), un problema que la indexación tradicional sobre columnas de filtrado no puede resolver por sí sola.

Etapa del plan	**Operación / Índice utilizado**	**Filas (actual)**
Filtro de órdenes por estado y fecha	Bitmap Index Scan usando `idx_ordenes_estado_fecha` (seguido de Bitmap Heap Scan)	22,534
Join con DetalleOrden	Parallel Hash Join con Parallel Seq Scan sobre `detalleorden`	67,779
Join Productos–DetalleOrden	Nested Loop con Index Scan usando `productos_pkey`	67,779
Join Vendedores–Productos	Hash Join con Seq Scan sobre `vendedores`	67,779
Join (Right Join) con Resenas	Parallel Hash Right Join con Parallel Seq Scan sobre `resenas`	67,779
Agregación y ordenamiento final	GroupAggregate, Sort y Limit	5

Tabla 36: Resumen del plan de ejecución de la Consulta (Sección 5.3.3)

Métrica del plan	Primer Intento (Caché Fría)	Intentos Post. (Caché Caliente)
Buffers *shared hit* (RAM)	265,329 bloques	315,593 bloques
Buffers *shared read* (Disco)	50,264 bloques	0 bloques
Planning Time	4.755 ms	1.150 ms
Execution Time (Promedio)	1,786.415 ms	1,490.194 ms

Tabla 37: Comparativa de métricas clave del plan de ejecución, Consulta 3 con índices (1M registros)

5.3.4. Planes de índices para Consulta 4

Para optimizar la Consulta 4 se crearon 4 índices, justificados por las columnas utilizadas en las cláusulas `WHERE` y `JOIN`:

```
CREATE INDEX idx_cupones_vencimiento ON Cupones(fecha_vencimiento);
CREATE INDEX idx_productos_categoria_stock ON Productos(id_categoria, stock);
CREATE INDEX idx_ordenes_cupon ON Ordenes(id_cupon);
CREATE INDEX idx_detalleorden_producto ON DetalleOrden(id_producto);
```

Plan sin índices (escenario 1,000,000 registros): en este escenario, antes de crear los índices, PostgreSQL resuelve la consulta mediante *Seq Scan* sobre `ordenes` (1,000,000 filas) y *Nested Loop* con `detalleorden`, alcanzando un tiempo de ejecución promedio de 193.04 ms (ver Sección 5.5.1).

Plan con índices (escenario 1,000,000 registros): tras crear los índices, el plan de ejecución confirma que los 4 índices son efectivamente utilizados por el optimizador:

- `idx_productos_categoria_stock` se usa mediante *Index Only Scan* en la subconsulta que determina la categoría con más quiebres de stock, y mediante *Bitmap Heap Scan* en el join principal con `Productos`.
- `idx_ordenes_cupon` se usa mediante *Parallel Index Scan* para el *Merge Join* entre `Ordenes` y `Cupones`.
- `idx_cupones_vencimiento` se usa mediante *Bitmap Index Scan* para filtrar los cupones por fecha de vencimiento.

A pesar de que los índices se utilizan correctamente, el tiempo de ejecución solo mejoró un 2 % (de 193.04 ms a 189.02 ms). Esto se debe a que el costo dominante de la consulta no proviene de la búsqueda puntual de filas (lo que el índice optimiza), sino del volumen de filas que deben recorrerse en el *Nested Loop* entre `Ordenes` y `DetalleOrden`: el plan muestra 79,496 iteraciones del *Index Only Scan* sobre `detalleorden_pkey`, generando 478,081 lecturas de bloques de caché (*shared hit*) y 60,640 lecturas de disco (*shared read*), equivalentes a aproximadamente 4.2 GB de datos procesados. Este comportamiento evidencia que un índice reduce el costo de acceso puntual a los datos, pero no puede reducir el costo de procesar un resultado intermedio genuinamente grande, que es inherente a la cardinalidad de la consulta en el escenario de un millón de registros.

Etapa del plan	Operación / Índice utilizado	Filas (actual)
Subconsulta (categoría con más quiebres)	Index Only Scan usando `idx_productos_categoria_stock`	24,075
Filtro de cupones por vencimiento	Bitmap Index Scan usando `idx_cupones_vencimiento`	1,590
Join Ordenes–Cupones	Parallel Index Scan usando `idx_ordenes_cupon` (Merge Join)	83,259
Join Productos–Categoría filtrada	Parallel Bitmap Heap Scan usando `idx_productos_categoria_stock`	12,468
Join final Ordenes–DetalleOrden	Index Only Scan usando `detalleorden_pkey` (Nested Loop, 79,496 iteraciones)	29,814

Tabla 38: Resumen del plan de ejecución de la Consulta 4 con índices, escenario de 1,000,000 registros

Métrica del plan	Valor
Buffers *shared hit* (caché)	478,081 bloques
Buffers *shared read* (disco)	60,640 bloques
Datos procesados (aprox.)	4.2 GB
Planning Time	1.66 ms
Execution Time	184.17 ms

Tabla 39: Métricas clave del plan de ejecución, Consulta 4 con índices (1M registros)

5.4. Plataforma de Pruebas

Las pruebas de experimentación de la Consulta 1 y 2 se realizaron en el siguiente entorno:

- **Equipo**: ASUS VivoBook (11thGen Intel Core i5).
- **Sistema operativo**: Windows 11.
- **RAM**: 16 GB.
- **Capacidad SSD**: 954 GB.
- **Motor de base de datos**: PostgreSQL 18.3.
- **Herramienta de generación de datos**: Python 3.9 con las librerías `Faker` 37.12.0 y `psycopg2-binary` 2.9.12.
- **Herramienta de prueba de consultas**: pgAdmin4 (PostgreSQL), mediante *Query Tool* conectada a 4 bases de datos PostgreSQL independientes, una por escenario de volumen (`falabella_1k`, `falabella_10k`, `falabella_100k`, `falabella_1m`).

Las pruebas de experimentación de la Consulta 4 se realizaron en el siguiente entorno:

- **Equipo**: MacBook Pro (chip Apple M4 Pro).

- **Sistema operativo**: macOS.
- **RAM**: 24 GB.
- **Capacidad SSD**: 512 GB.
- **Motor de base de datos**: PostgreSQL 18.3.
- **Herramienta de generación de datos**: Python 3.9 con las librerías `Faker` 37.12.0 y `psycopg2-binary` 2.9.12.
- **Herramienta de prueba de consultas**: DataGrip (JetBrains), mediante *Query Console* conectada a 4 bases de datos PostgreSQL independientes, una por escenario de volumen (`falabella_1k`, `falabella_10k`, `falabella_100k`, `falabella_1m`).

5.5. Medición de tiempos

5.5.1. Sin índices

Consulta 1

Ejecución	1K	10K	100K	1M
1	2.387	30.235	510.932	2868.916
2	2.852	25.215	523.860	2990.741
3	3.384	28.363	798.522	3235.256
4	2.236	33.450	480.468	3035.345
5	2.213	28.145	503.853	3179.245
Promedio	**2.614**	**29.082**	**563.527**	**3061.901**

Tabla 40: Consulta 1 – Tiempos de ejecución (ms) sin índices, 5 corridas por escenario

Consulta 2

Ejecución	1K	10K	100K	1M
1	0.841	8.777	11.450	197.143
2	1.517	5.520	16.780	305.264
3	1.403	7.795	10.105	262.021
4	1.440	8.532	9.777	186.164
5	1.304	5.727	12.593	190.993
Promedio	**1.301**	**7.270**	**12.141**	**228.317**

Tabla 41: Consulta 2 – Tiempos de ejecución (ms) sin índices, 5 corridas por escenario

Consulta 3

Ejecución	1K	10K	100K	1M
1	2.342	22.062	582.529	1786.415
2	2.099	17.233	248.454	1708.678
3	2.908	19.214	250.539	1651.444
4	2.380	16.900	232.629	1511.927
5	2.345	17.532	248.624	1498.919
Promedio	**2.415**	**18.588**	**312.555**	**1631.477**

Tabla 42: Consulta 3 – Tiempos de ejecución (ms) sin índices, 5 corridas por escenario

Consulta 4

Ejecución	1K	10K	100K	1M
1	1.004	21.406	102.332	267.052
2	1.004	8.981	52.383	184.735
3	1.239	9.395	48.593	169.987
4	1.645	10.685	47.686	173.758
5	0.843	5.858	49.332	169.672
Promedio	**1.147**	**11.265**	**60.065**	**193.041**

Tabla 43: Consulta 4 – Tiempos de ejecución (ms) sin índices, 5 corridas por escenario

5.5.2. Con índices

Consulta 1

Ejecución	1K	10K	100K	1M
1	2.694	20.423	392.019	2653.178
2	2.118	24.151	432.509	2692.194
3	2.036	20.857	393.805	3037.776
4	3.351	22.931	419.392	3004.558
5	2.042	25.977	410.376	3202.602
Promedio	**2.448**	**22.868**	**409.620**	**2918.062**

Tabla 44: Consulta 1 – Tiempos de ejecución (ms) con índices, 5 corridas por escenario

Consulta 2

Ejecución	1K	10K	100K	1M
1	1.473	5.718	55.043	66.253
2	0.768	6.024	52.738	66.487
3	0.860	8.781	44.583	68.421
4	1.491	5.222	52.411	60.773
5	1.365	5.106	49.452	66.027
Promedio	**1.191**	**6.170**	**50.845**	**65.592**

Tabla 45: Consulta 2 – Tiempos de ejecución (ms) con índices, 5 corridas por escenario

Consulta 3

Ejecución	1K	10K	100K	1M
1	2.293	16.023	203.138	1563.227
2	2.855	14.256	192.781	1559.574
3	2.110	14.888	203.127	1482.713
4	1.746	17.117	202.144	1492.637
5	2.294	14.818	200.253	1352.840
Promedio	**2.260**	**15.420**	**200.289**	**1490.198**

Tabla 46: Consulta 3 – Tiempos de ejecución (ms) con índices, 5 corridas por escenario

Consulta 4

Ejecución	1K	10K	100K	1M
1	2.620	10.657	65.583	211.815
2	2.150	7.251	33.959	181.913
3	1.574	6.939	44.034	180.942
4	0.976	7.427	38.388	181.414
5	0.874	5.827	40.263	189.026
Promedio	**1.639**	**7.620**	**44.445**	**189.022**

Tabla 47: Consulta 4 – Tiempos de ejecución (ms) con índices, 5 corridas por escenario

5.6. Resultados

5.6.1. Consulta 1

Escenario	Sin índices (ms)	Con índices (ms)	Variación
1,000	2.614	2.448	+6.35 %
10,000	29.082	22.868	+21.37 %
100,000	563.527	409.620	+27.31 %
1,000,000	3061.901	2918.062	+4.70 %

Tabla 48: Tiempos de ejecución promedio (5 corridas) de la Consulta 1, con y sin índices, por escenario

Los resultados consolidados para la Consulta 2 muestran un comportamiento asintótico y dispar respecto a la efectividad de las estructuras de indexación introducidas. A diferencia de otras consultas del estudio, el impacto de los índices se concentra de forma favorable en las escalas intermedias de 10,000 y 100,000 registros, donde se obtuvieron reducciones en el tiempo de procesamiento del 21.37 % y 27.31 % respectivamente. En este rango, los índices lógicos insertados en las columnas críticas facilitan los acoplamientos internos, aliviando la carga computacional asociada a la indexación de las llaves foráneas.

Sin embargo, el hallazgo más representativo de este experimento radica en el escenario masivo de 1,000,000 de registros, donde la ganancia de rendimiento con índices disminuyó drásticamente a un marginal 4.70 %. Este fenómeno ocurre porque el verdadero cuello de botella de la Consulta 2 no reside en la búsqueda o el filtrado de registros aislados, sino en el alto costo computacional de la subconsulta anidada. Al requerir el cálculo del promedio global de la columna `subtotal` evaluando la totalidad de la tabla `DetalleOrden` (operación matemática que se multiplica por dos), el optimizador se ve obligado a realizar agregaciones masivas sobre millones de filas.

Dado que las funciones de agregación globales de tipo *Aggregate* o *HashAggregate* requieren un escaneo físico completo de la estructura de datos para computar los valores matemáticos, los índices tradicionales sobre columnas de filtro pierden efectividad a gran escala. En consecuencia, el tiempo invertido por la CPU en la resolución aritmética de la subconsulta predomina sobre el tiempo de acceso a los bloques de disco, demostrando que en consultas de alta complejidad analítica y agregaciones masivas, la optimización mediante índices de búsqueda tradicionales alcanza un límite de rendimiento condicionado por el diseño lógico de la misma consulta.

5.6.2. Consulta 2

Escenario	Sin índices (ms)	Con índices (ms)	Variación
1,000	1.301	1.191	+8.45 %
10,000	7.270	6.170	+15.13 %
100,000	12.141	50.845	-318.79 % (empeoró)
1,000,000	228.317	65.592	+71.27 %

Tabla 49: Tiempos de ejecución promedio (5 corridas) de la Consulta 2, con y sin índices, por escenario

Los resultados consolidados revelan un comportamiento altamente dinámico y no lineal respecto a la utilidad de las estructuras de indexación, determinado directamente por el volumen de la carga de datos. En las escalas menores (1,000 y 10,000 registros), la consulta experimentó optimizaciones modestas de entre 8.45 % y 15.13 % respectivamente, debido a que el motor ya resolvía las operaciones secuenciales con latencias mínimas en memoria RAM.

Sin embargo, el escenario de 100,000 registros presentó una anomalía crítica donde el uso de índices resultó contraproducente, incrementando el tiempo de respuesta en un 318.79 %. Este comportamiento se fundamenta en el fenómeno de *Index Overhead*: en este volumen

intermedio de información, el costo computacional que le toma a PostgreSQL procesar el árbol B-Tree del índice y realizar saltos lógicos secuenciales supera con creces el beneficio frente a un barrido directo por bloques (*Seq Scan*), el cual procesa las páginas físicas de corrido dentro de la memoria caché.

Por el contrario, en el escenario crítico de 1,000,000 de registros, los índices demostraron su verdadero poder de escalabilidad, alcanzando una mejora drástica del 71.27 % al desplomar el tiempo de ejecución de 228.32 ms a tan solo 65.59 ms. A gran escala, la alta selectividad de la regla de negocio (productos con precios superiores al doble del promedio global) permite que el planificador evite escaneos secuenciales masivos sobre millones de páginas en disco. Los índices transforman los costosos acoplamientos tipo *Hash Join* en eficientes búsquedas indexadas directas (*Index Scan*), logrando mitigar el impacto del crecimiento del volumen de datos y estabilizando el consumo de recursos en la caché caliente de la base de datos.

5.6.3. Consulta 3

Escenario	Sin índices (ms)	Con índices (ms)	Variación
1,000	2.607	2.399	+7.98 %
10,000	22.209	16.049	+27.74 %
100,000	325.979	211.724	+35.05 %
1,000,000	1631.480	1490.194	+8.66 %

Tabla 50: Tiempos de ejecución promedio (5 corridas) de la Consulta 3, con y sin índices, por escenario

Los resultados consolidados para la Consulta 3 muestran un comportamiento en forma de campana en cuanto a la efectividad de las estructuras de indexación. En el escenario inicial de 1,000 registros, la ganancia es modesta (+7.98 %), ya que el bajo volumen permite al motor resolver los cruces rápidamente en memoria, limitando la utilidad de acceder a los árboles de los índices. A medida que la carga asciende a los escenarios intermedios de 10,000 y 100,000 registros, los índices demuestran su máxima rentabilidad, alcanzando mejoras significativas del 27.74 % y 35.05 % respectivamente. En este segmento, el costo de recorrer los índices compensa ampliamente la sobrecarga de un escaneo secuencial.

Sin embargo, los resultados en el escenario de estrés extremo de 1,000,000 de registros revelan un comportamiento crítico sobre los límites operativos de la indexación frente a planes de ejecución complejos. En este punto, la curva de mejora sufre una regresión drástica, consolidando una ganancia de rendimiento moderada de apenas un 8.66 % (descendiendo de 1,631.480 ms a 1,490.194 ms).

El análisis detallado del planificador de PostgreSQL demuestra que, a pesar de haberse diseñado cuatro estructuras de indexación para cubrir las cláusulas `WHERE` y `JOIN`, el optimizador tomó la decisión estratégica de descartar tres de ellas a gran escala. Únicamente el índice `idx_ordenes_estado_fecha` fue seleccionado a través de un *Bitmap Index Scan*, logrando aislar eficientemente las tuplas que cumplían con las restricciones de estado y fecha, evitando un barrido masivo sobre `Ordenes`.

Por el contrario, las estructuras `idx_productos_vendedor`, `idx_detalleorden_producto` y `idx_resenas_producto_orden` fueron completamente ignoradas. Para la tabla `Vendedores` (2,000 registros), un barrido secuencial directo en memoria resultó matemáticamente más económico que asumir la sobrecarga de Entrada/Salida del índice. A gran escala, para tablas de alto crecimiento como `DetalleOrden` y `Resenas`, el volumen masivo de registros intermedios obligó al motor a procesar los acoplamientos mediante algoritmos de *Parallel Hash Join*. Para alimentar de forma óptima estas tablas Hash en la memoria RAM, PostgreSQL determinó que un *Parallel Seq Scan* multifilo ofrecía un rendimiento sustancialmente superior frente a la latencia por accesos aleatorios al disco (*Random Disk Access*) que implicaría el uso de índices.

En consecuencia, el plan de ejecución retuvo los barridos secuenciales como estrategia dominante. Este comportamiento empírico demuestra que el verdadero cuello de botella de la Consulta 3 no reside en la selectividad del filtrado inicial, sino en el costo estructural e inevitable de los *joins* encadenados entre cinco tablas masivas y el volumen de filas resultantes que deben ordenarse en memoria (*top-N heapsort*); un desafío cuya carga computacional no puede ser disipada únicamente mediante índices tradicionales.

5.6.4. Consulta 4

Escenario	Sin índices (ms)	Con índices (ms)	Variación
1,000	1.15	1.64	-43 % (empeoró)
10,000	11.27	7.62	+32 %
100,000	60.07	44.45	+26 %
1,000,000	193.04	189.02	+2 %

Tabla 51: Tiempos de ejecución promedio (5 corridas) de la Consulta 4, con y sin índices, por escenario

Los resultados muestran un comportamiento no monótono respecto a la utilidad de los índices: en el escenario de 1,000 registros, el uso de índices fue contraproducente, ya que el costo de mantener y recorrer la estructura del índice superó el beneficio frente a un simple *Seq Scan* sobre una tabla pequeña — comportamiento que el propio optimizador de PostgreSQL ya anticipaba en el plan sin índices de este escenario, donde eligió *Seq Scan* aun teniendo la posibilidad de no hacerlo. En los escenarios intermedios (10,000 y 100,000), los índices aportaron una mejora notoria (entre 26 % y 32 %), rango en el cual el volumen de datos ya es suficiente para justificar el costo del índice pero aún no domina el costo del *join* entre tablas. En el escenario de 1,000,000, la mejora se redujo a un 2 %, ya que el cuello de botella pasó a ser el volumen de filas procesadas en el *Nested Loop* entre `Ordenes` y `DetalleOrden` (ver análisis detallado en la Sección 5.3.4), un costo que ningún índice sobre columnas de filtro puede reducir por sí solo.

5.7. Análisis y Discusión

El análisis consolidado de la plataforma de experimentación sobre el ecosistema transaccional de Falabella Perú (evaluado en escalas de 1,000, 10,000, 100,000 y 1,000,000 de registros)

depara comportamientos asintóticos y dispares en la efectividad de las estructuras de indexación física. Los resultados demuestran que el rendimiento no depende únicamente de la presencia de índices, sino de la semántica de la consulta, la selectividad de los predicados y la naturaleza de los algoritmos de acoplamiento del motor PostgreSQL.

5.7.1. Consulta 1: Análisis Técnico de la Anomalía Semántica e Impacto de *Fan-out*

La Consulta 1 fue diseñada originalmente para identificar clientes de alto valor con inversiones duplicadas respecto al promedio y calificaciones críticas. No obstante, una auditoría detallada en la fase de post-producción reveló una anomalía semántica crítica en la condición de acoplamiento (`JOIN`) de la entidad `Resenas`. Al enlazar `DetalleOrden` con `Resenas` exclusivamente bajo el criterio `d.id_producto = r.id_producto`, se omitió la restricción relacional que vincula la reseña con la orden específica del cliente (`AND r.id_orden = o.id_orden`).

Desde la perspectiva del optimizador de consultas, este error lógico forzó al motor a generar un volumen masivo de tuplas intermedias artificiales, provocando un fenómeno de producto cartesiano parcial o efecto *fan-out*. Cada línea de detalle de orden se multiplicó por la cardinalidad total de reseñas asociadas a ese producto en la base de datos. A la escala masiva de 1,000,000 de registros, este factor multiplicador redundante saturó los operadores de agregación en las cláusulas `GROUP BY` y `HAVING`. Como consecuencia, el cuello de botella se trasladó por completo al uso intensivo de la CPU para procesar y consolidar filas espurias en memoria, lo que explica por qué la introducción de índices físicos arrojó una mejora marginal de apenas el **2 %**. Este hallazgo subraya que una imprecisión en el diseño del modelo lógico-semántico invalida la optimización física y distorsiona métricas de negocio críticas como el monto total invertido (`SUM`) y la peor puntuación (`AVG`).

5.7.2. Consulta 2: Comportamiento Asintótico y Degradación por Volumen de Filas

Los resultados experimentales de la Consulta 2 evidencian que el impacto de la indexación se concentra de forma óptima en las escalas intermedias de la base de datos. En los escenarios de 10,000 y 100,000 registros, se lograron reducciones sustanciales en el tiempo de procesamiento de **21.37 %** y **27.31 %** respectivamente. En estos entornos, los índices lógicos insertados mitigaron eficientemente el costo analítico de los acoplamientos y agilizaron el acceso a través de las llaves foráneas.

Sin embargo, al migrar al escenario masivo de 1,000,000 de registros, la mejora de rendimiento decayó drásticamente a un marginal **2 %**. El plan de ejecución arrojado por `EXPLAIN ANALYZE` demostró que el cuello de botella estructural se trasladó al volumen neto de filas procesadas mediante el algoritmo de bucle anidado (`Nested Loop`) entre las tablas `Ordenes` y `DetalleOrden`. A esta escala extrema, el costo inherente de iterar y unir conjuntos de datos gigantescos domina por completo la consulta, haciendo que los índices sobre columnas de filtrado pierdan tracción operativa si no se acompaña la estrategia con particionamiento físico o reescritura de consultas.

5.7.3. Consulta 3: Inocuidad de Índices en Consultas Complejas Acotadas por CPU

El experimento con la Consulta 3 arrojó un hallazgo interesante: el optimizador de PostgreSQL ignoró por completo múltiples estructuras físicas creadas, específicamente los índices `idx_productos_vendedor`, `idx_detalleorden_producto` y `idx_resenas_producto_orden`. Para la tabla `Vendedores` (acotada jerárquicamente a 2,000 registros), el motor determinó que realizar un barrido secuencial directo en memoria RAM (`Seq Scan`) era matemáticamente más económico que asumir la sobrecarga de Entrada/Salida (I/O) que impone la lectura y recorrido de un árbol B+.

A la escala de un millón de registros, las tablas de alta tasa de crecimiento como `DetalleOrden` y `Resenas` generaron tal volumen de datos intermedios que el motor descartó los accesos aleatorios al disco (*Random Disk Access*) inducidos por los índices. En su lugar, aplicó algoritmos de reunión por hash paralela (`Parallel Hash Join`) alimentados por `Parallel Seq Scan` multifilo para optimizar el ancho de banda del hardware. Como consecuencia, el tiempo de ejecución promedio global solo disminuyó un **4.70 %** (de 3061.90 ms a 2918.06 ms). Esto ratifica que el costo dominante es de naturaleza puramente analítica (operaciones de agregación en el `HAVING` y resolución de subconsultas correlacionadas complejas), manteniendo la consulta acotada por CPU.

5.7.4. Consulta 4: Éxito Rotundo del Filtrado Indexado y Alta Selectividad

En contraste directo con la Consulta 3, la Consulta 4 demostró la máxima expresión de eficiencia de un plan de optimización física bien direccionado. En el escenario base sin índices, el motor se vio obligado a ejecutar un `Seq Scan` masivo sobre el millón de filas de la tabla `Ordenes` y encadenar un `Nested Loop` altamente costoso, promediando un tiempo de **193.04 ms**.

Al introducir los índices compuestos y específicos, el plan de ejecución se transformó radicalmente: implementó un `Index Only Scan` restrictivo sobre el índice `idx_productos_categoria_stock` que aisló velozmente las 24,075 filas pertinentes a la subconsulta de categorías en quiebre, combinado con un `Bitmap Index Scan` en `idx_cupones_vencimiento` para filtrar apenas 1,590 registros lógicos. La resolución de los acoplamientos mayores se procesó mediante un `Parallel Index Scan` bajo la estrategia de `Merge Join` para unir `Ordenes` y `Cupones` (83,259 filas lógicas). Este comportamiento demuestra que cuando los predicados poseen una alta selectividad y los índices permiten realizar búsquedas directas o escaneos puramente indexados en memoria, la reducción del espacio de búsqueda mitiga drásticamente la latencia de procesamiento, validando el diseño físico para consultas de alta precisión.

6. Conclusiones

Tras el diseño, implementación y evaluación experimental del modelo relacional en PostgreSQL para el ecosistema transaccional de Falabella Perú, se formulan las siguientes conclusiones fundamentales del proyecto:

1. **Dependencia del Contexto y la Escala en la Indexación:** Se concluye que la creación de índices no garantiza una aceleración lineal ni universal del rendimiento. Su efectividad está estrictamente supeditada a la cardinalidad y tamaño físico de las tablas en disco; el planificador priorizará escaneos secuenciales (`Seq Scan`) en volúmenes pequeños (1,000 registros) debido al bajo costo de lectura de páginas en memoria, mientras que activará escaneos indexados (`Index Scan`) en volúmenes intermedios (10,000 y 100,000 registros), logrando eficiencias de entre el 21 % y 32 % de mejora (Consultas 2 y 4).
2. **Vulnerabilidad de la Optimización Física ante Defectos Semánticos:** La auditoría técnica realizada post experimento sobre la Consulta 1 demostró que un error en el plano lógico semántico como la omisión de llaves correlacionales al unir la tabla `Resenas` anula las ventajas del diseño físico. Al gatillar un fenómeno de *fan out* o multiplicación exponencial de tuplas redundantes en memoria, el cuello de botella se desplaza hacia los operadores de ordenamiento y agregación en la CPU, reduciendo la ganancia de los índices a un marginal 2 % en el escenario de 1,000,000 de registros.
3. **Inocuidad Estructural en Consultas Acotadas por CPU:** El comportamiento de la Consulta 3 evidencia que en consultas complejas de naturaleza analítica (operaciones pesadas de agrupamiento, cláusulas `HAVING` y subconsultas complejas), el optimizador de PostgreSQL tiende a descartar de manera autónoma los índices creados sobre tablas masivas. Para mitigar la latencia, el motor prefiere algoritmos de `Parallel Hash Join` y barridos secuenciales paralelos, demostrando que el costo dominante es computacional y no de acceso a disco, limitando la mejora a un leve 4.70 %.
4. **Efectividad de Índices Compuestos ante Predicados de Alta Selectividad:** Los resultados de la Consulta 4 confirman que la indexación física alcanza su máximo rendimiento cuando los predicados de filtro restringen severamente el espacio de búsqueda. El uso coordinado de índices específicos permitió al planificador sustituir lecturas masivas de tablas por escaneos de solo índice (`Index Only Scan`) y escaneos de mapas de bits (`Bitmap Index Scan`), aislando miles de filas sobre un universo de un millón y reduciendo la latencia computacional de manera óptima.
5. **Validación del Diseño de Datos y Atributos Derivados:** Se concluye que la inclusión estratégica de los campos redundantes calculados `precio_unitario` y `subtotal` en la entidad débil `DetalleOrden` es una solución arquitectónica indispensable. Además de blindar la invariabilidad histórica y la consistencia de las auditorías comerciales de Falabella Perú ante fluctuaciones de precios en el catálogo de productos, previene la sobrecarga computacional de tener que recalcular dichos agregados financieros en consultas transaccionales de gran escala.

7. Anexos

- Video de experimentación, Consulta 1, escenario de 1,000,000 registros (sin y con índices): `https://drive.google.com/file/d/1XSa8QxNXgZhFXROmWkwiNidXfms2DrLH/view?usp=sharing`

- Video de experimentación, Consulta 2, escenario de 1,000,000 registros (sin y con índices): `https://drive.google.com/file/d/1wrXBQ4ofJSSUt3jhGIx235JP63CrqCrs/view?usp=sharing`
- Video de experimentación, Consulta 3, escenario de 1,000,000 registros (sin y con índices): `https://drive.google.com/file/d/1bEDyItamf97Og49j_uQQo_GVk0f5kvd6/view?usp=sharing`
- Video de experimentación, Consulta 4, escenario de 1,000,000 registros (sin y con índices): `https://drive.google.com/file/d/1W6ApM4zpPCEHuM3LQGZAbNSJsv8-68dh/view?usp=sharing`

7.1. Normalización de la base de datos

En esta sección se demuestra que el modelo relacional satisface la Tercera Forma Normal (3FN).

Clientes(id_cliente, nombre, apellido, email, telefono, fecha_registro, dni)
$F = \{id_cliente \rightarrow nombre, apellido, email, telefono, fecha_registro, dni\}$
`id_cliente` es superllave. La relación está en 3FN.

Categorias(id_categoria, nombre, descripcion)
$F = \{id_categoria \rightarrow nombre, descripcion\}$
`id_categoria` es superllave. La relación está en 3FN.

Vendedores(id_vendedor, nombre_tienda, email, ruc, calificacion)
$F = \{id_vendedor \rightarrow nombre_tienda, email, ruc, calificacion\}$
`id_vendedor` es superllave. La relación está en 3FN.

Productos(id_producto, nombre, descripcion, precio, stock, marca, tipo, id_categoria, id_vendedor)
$F = \{id_producto \rightarrow nombre, descripcion, precio, stock, marca, tipo, id_categoria, id_vendedor\}$
`id_producto` es superllave. La relación está en 3FN.

Cupones(id_cupon, codigo, descuento, tipo_descuento, fecha_vencimiento, usos_maximos, usos_actuales)
$F = \{id_cupon \rightarrow codigo, descuento, tipo_descuento, fecha_vencimiento, usos_maximos, usos_actuales\}$
`id_cupon` es superllave. La relación está en 3FN.

Ordenes(id_orden, fecha_orden, estado, total, id_cliente, id_cupon)
$F = \{id_orden \rightarrow fecha_orden, estado, total, id_cliente, id_cupon\}$
`id_orden` es superllave. La relación está en 3FN.

DetalleOrden(id_orden, id_producto, cantidad, precio_unitario, subtotal)
$F = \{id_orden, id_producto \rightarrow cantidad, precio_unitario, subtotal\}$
$(id_orden, id_producto)$ es superllave. La relación está en 3FN. El atributo `subtotal` es funcionalmente dependiente de `cantidad` y `precio_unitario`, pero al depender también de la llave primaria compuesta, no viola 3FN; se mantiene como atributo derivado almacenado por eficiencia.

Pagos(id_pago, metodo, estado, fecha_pago, monto, codigo_autorizacion, id_orden)
$F = \{id_pago \rightarrow metodo, estado, fecha_pago, monto, codigo_autorizacion, id_orden\}$
`id_pago` es superllave. La relación está en 3FN.

Envios(id_envio, direccion_destino, estado_envio, fecha_despacho, fecha_entrega, courier, numero_seguimiento, id_orden)
$F = \{id_envio \rightarrow direccion_destino, estado_envio, fecha_despacho, fecha_entrega, courier, numero_seguim$
`id_envio` es superllave. La relación está en 3FN.

Resenas(id_resena, puntuacion, comentario, fecha, id_cliente, id_orden, id_producto)
$F = \{id_resena \rightarrow puntuacion, comentario, fecha, id_cliente, id_orden, id_producto\}$
`id_resena` es superllave. La relación está en 3FN.

7.2. Dump Data y scripts

Los scripts de generación de datos (`generar_datos.py`) y los cuatro archivos de respaldo de la base de datos (`dump_1k.sql`, `dump_10k.sql`, `dump_100k.sql`, `dump_1m.sql`) están disponibles en el siguiente enlace:

`https://drive.google.com/drive/folders/1aXMBt1FmIXQLk0l3LPpcl5L-1KS1qXok?usp=sharing`